

Patrones de Diseño Orientado a Objetos

DISEÑO DE SISTEMAS DE SOFTWARE (2024)



Design Patterns

Strategy

Decorator

Factory

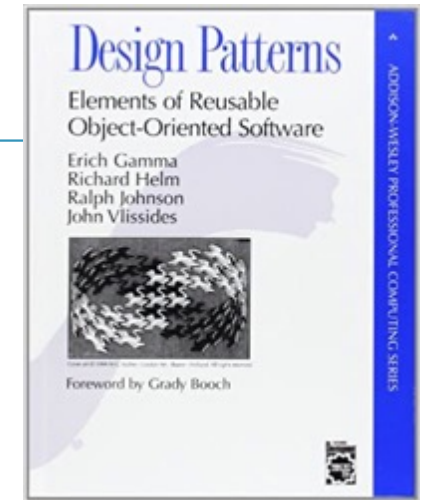
Singleton

Adapter

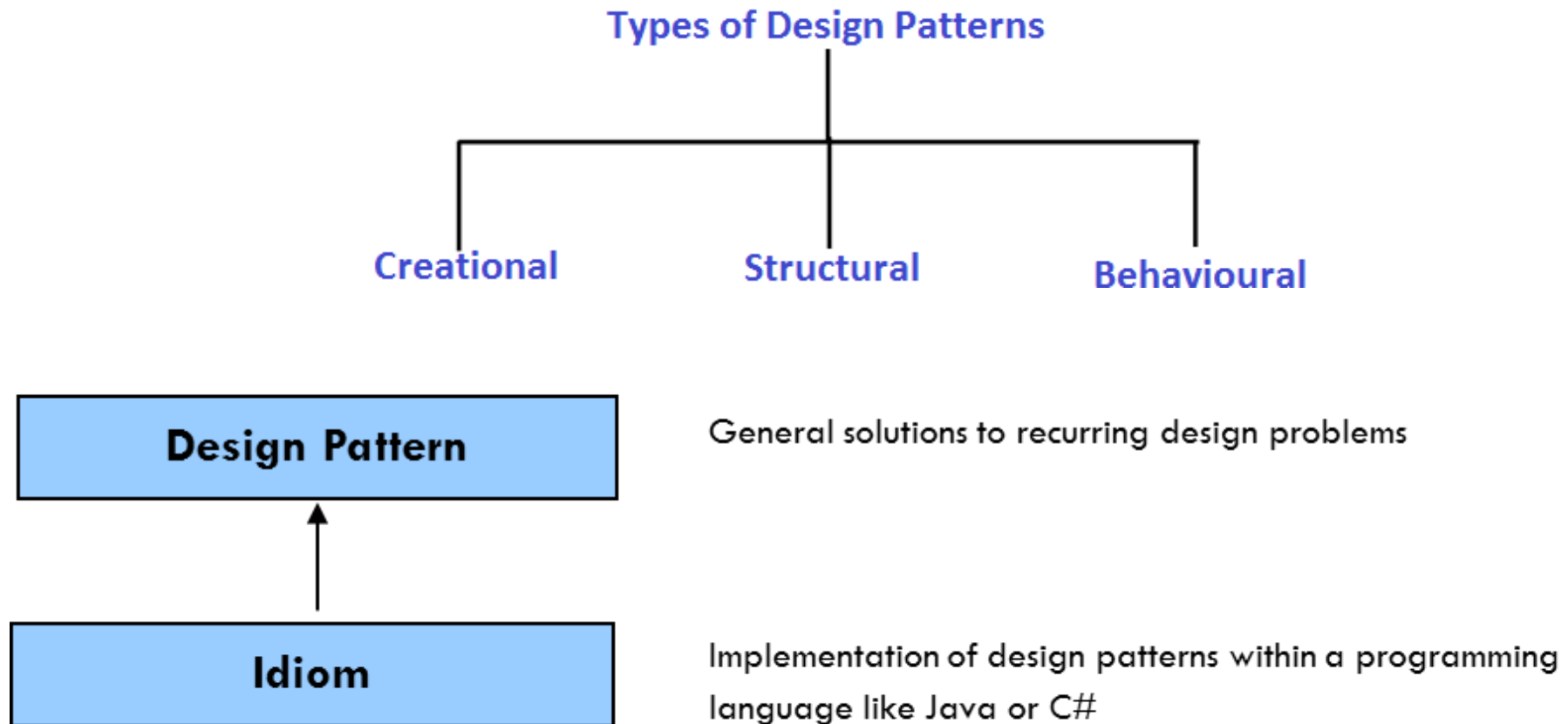
Command

Visitor

Template



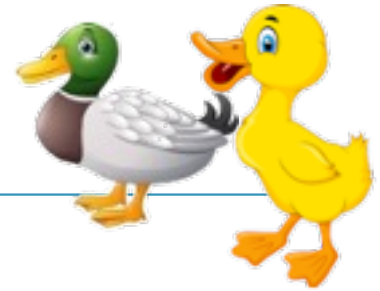
Tipos de Design Patterns e Idioms



Vamos a arrancar con un ejemplo...

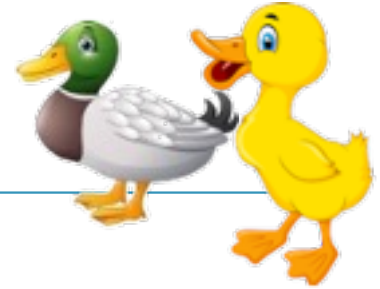
Pato, pato, ñato

- Se quiere hacer un simulador de patos
- Hay que modelar una gran variedad de patos, que nadan y hacen sonidos de patos

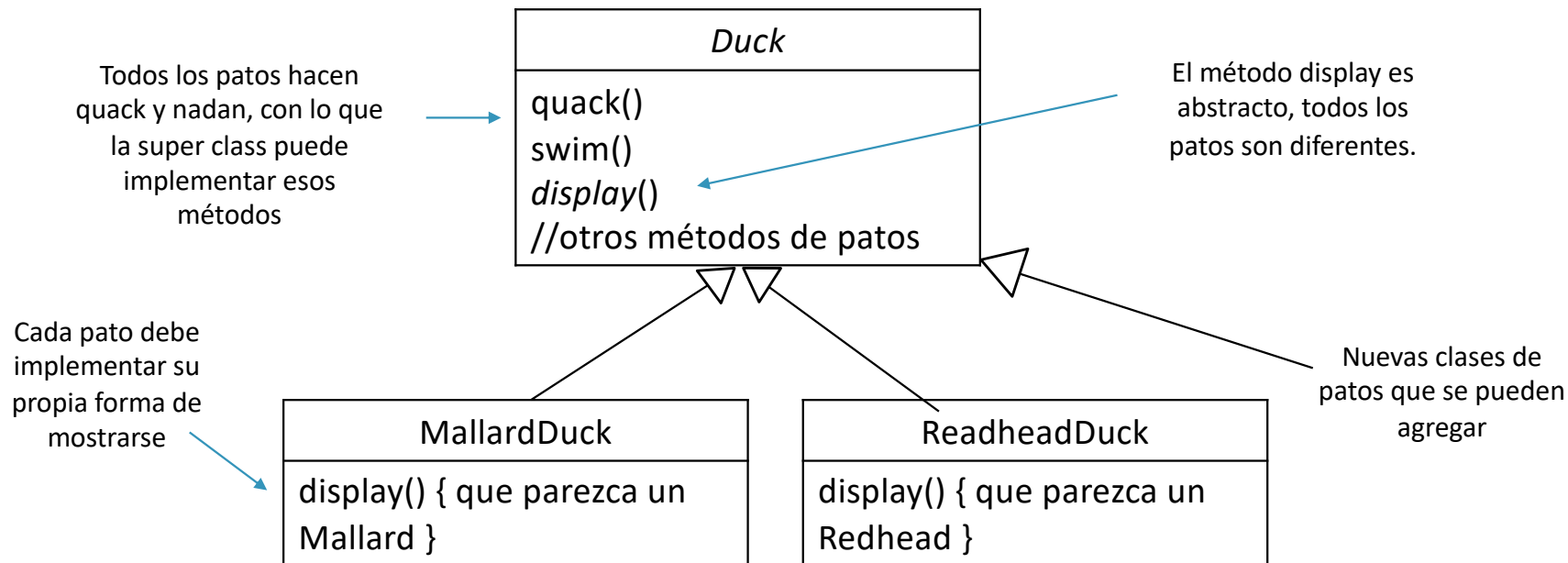


Vamos a arrancar con un ejemplo...

Pato, pato, ñato

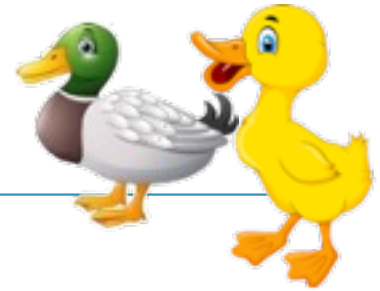


- Se quiere hacer un simulador de patos
- Hay que modelar una gran variedad de patos, que nadan y hacen sonidos de patos
- **Primera aproximación:** tener una super-clase Duck, de la cuál heredan los distintos patos



Vamos a arrancar con un ejemplo...

Pato, pato, ñato

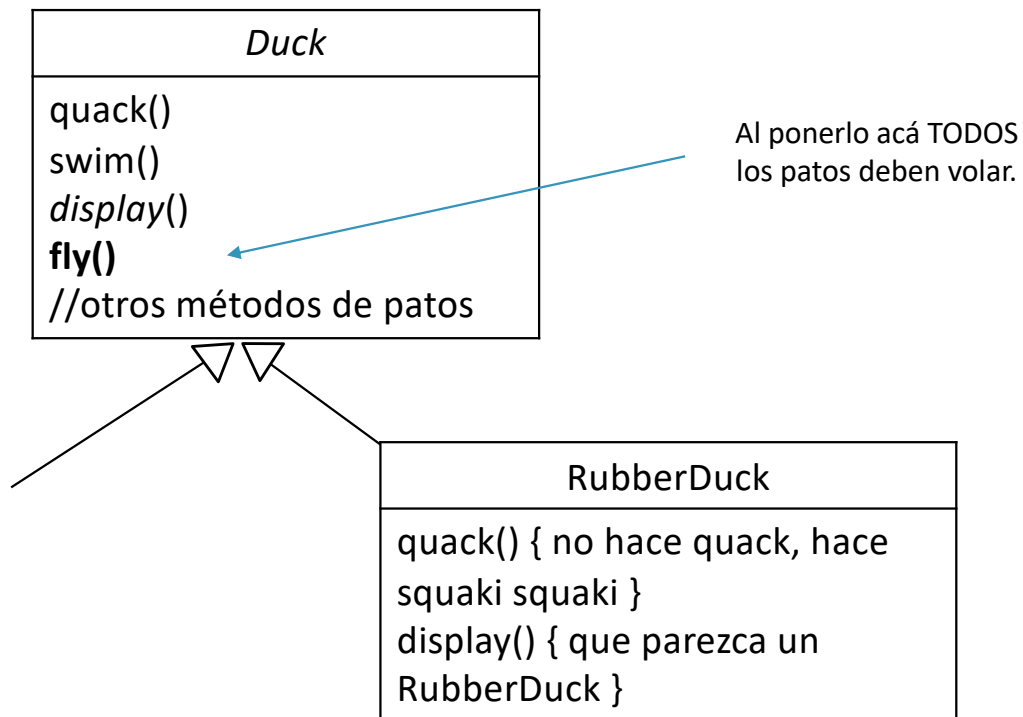


- Ahora queremos que los patos vuelen
- Agregamos el método en la clase pato

A wild rubber duck has appeared...

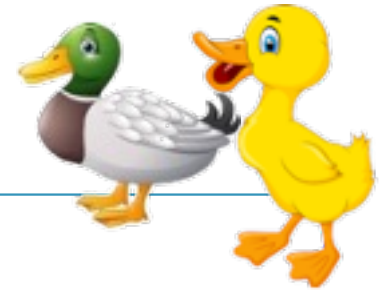


...volando!



- No todos las subclases de patos deben volar
- Al agregar el comportamiento en la superclase, se agregó también comportamiento que no es apropiado para algunas de las subclases
- Una actualización “localizada” del código causó un efecto no local

Vamos a arrancar con un ejemplo...

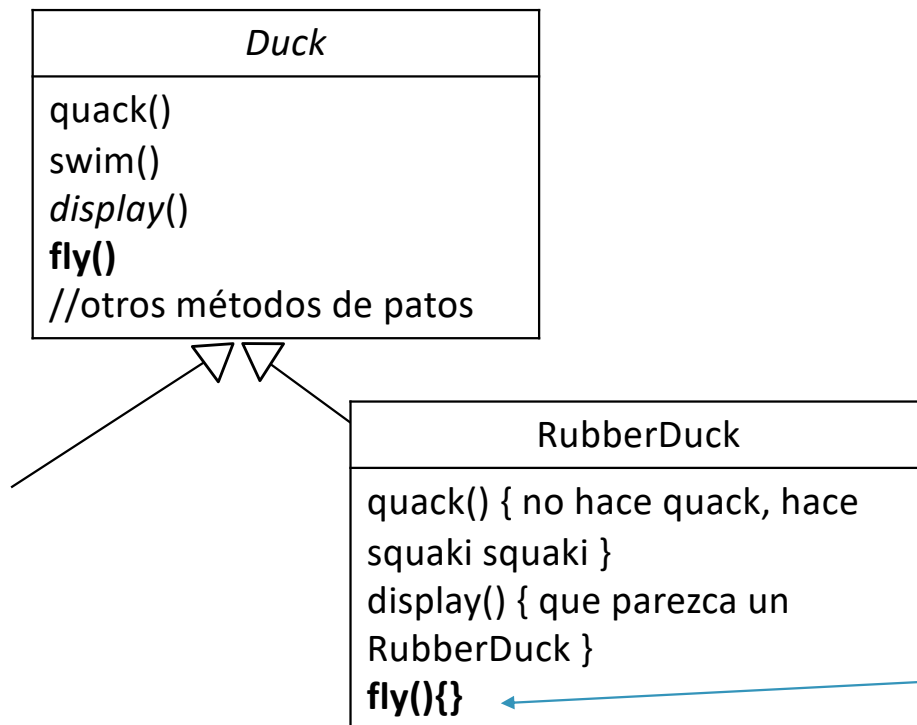


Pato, pato, ñato

- Ahora queremos que los patos vuelen
- Agregamos el método en la clase pato

A wild rubber duck has appeared...

...volando!



Se sobrescribe para que no haga nada

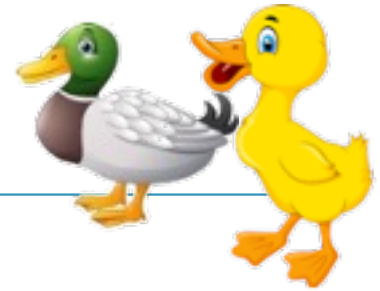
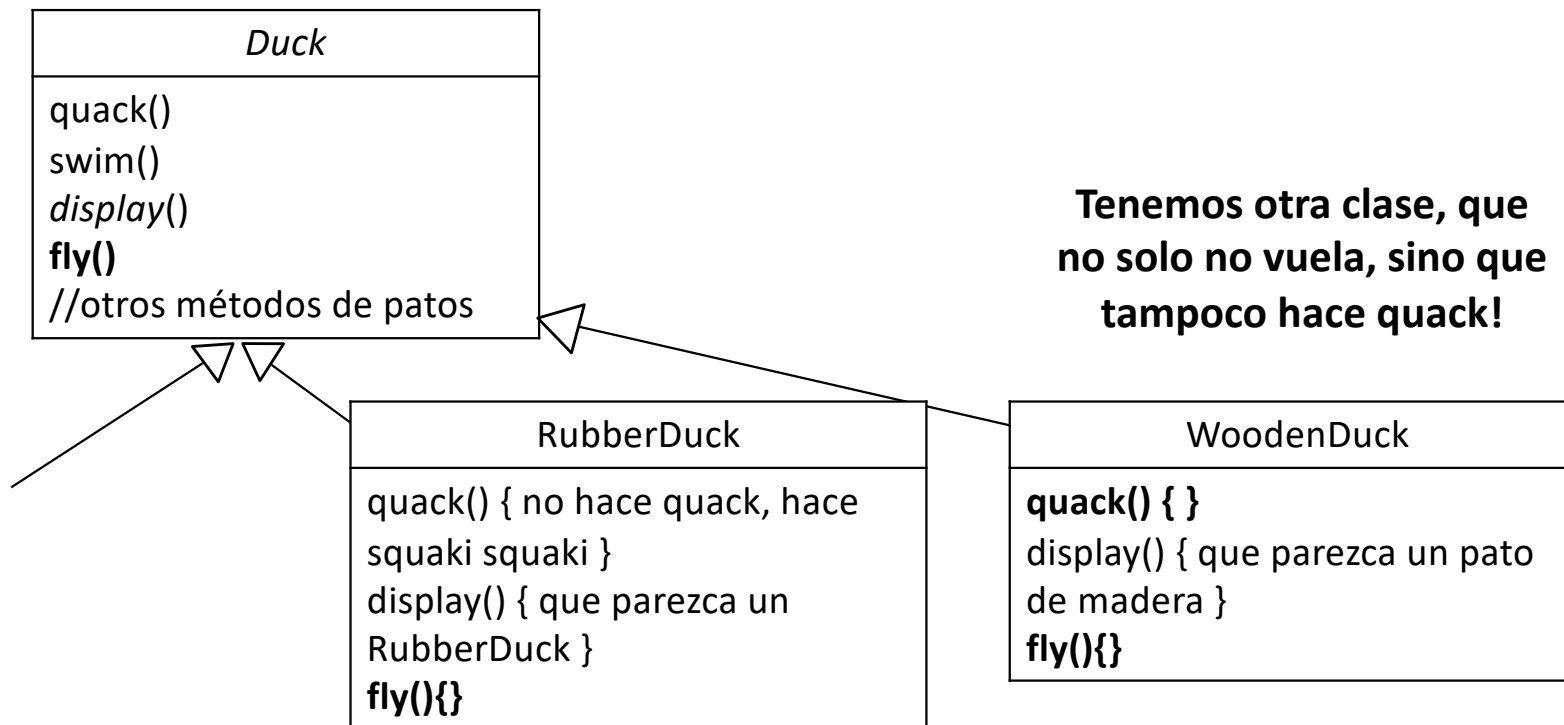
Vamos a arrancar con un ejemplo...

Pato, pato, ñato

- Ahora queremos que los patos vuelen
- Agregamos el método en la clase pato

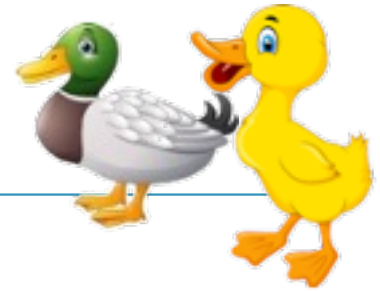
A wild wooden duck has appeared...

Tenemos otra clase, que no solo no vuela, sino que tampoco hace quack!



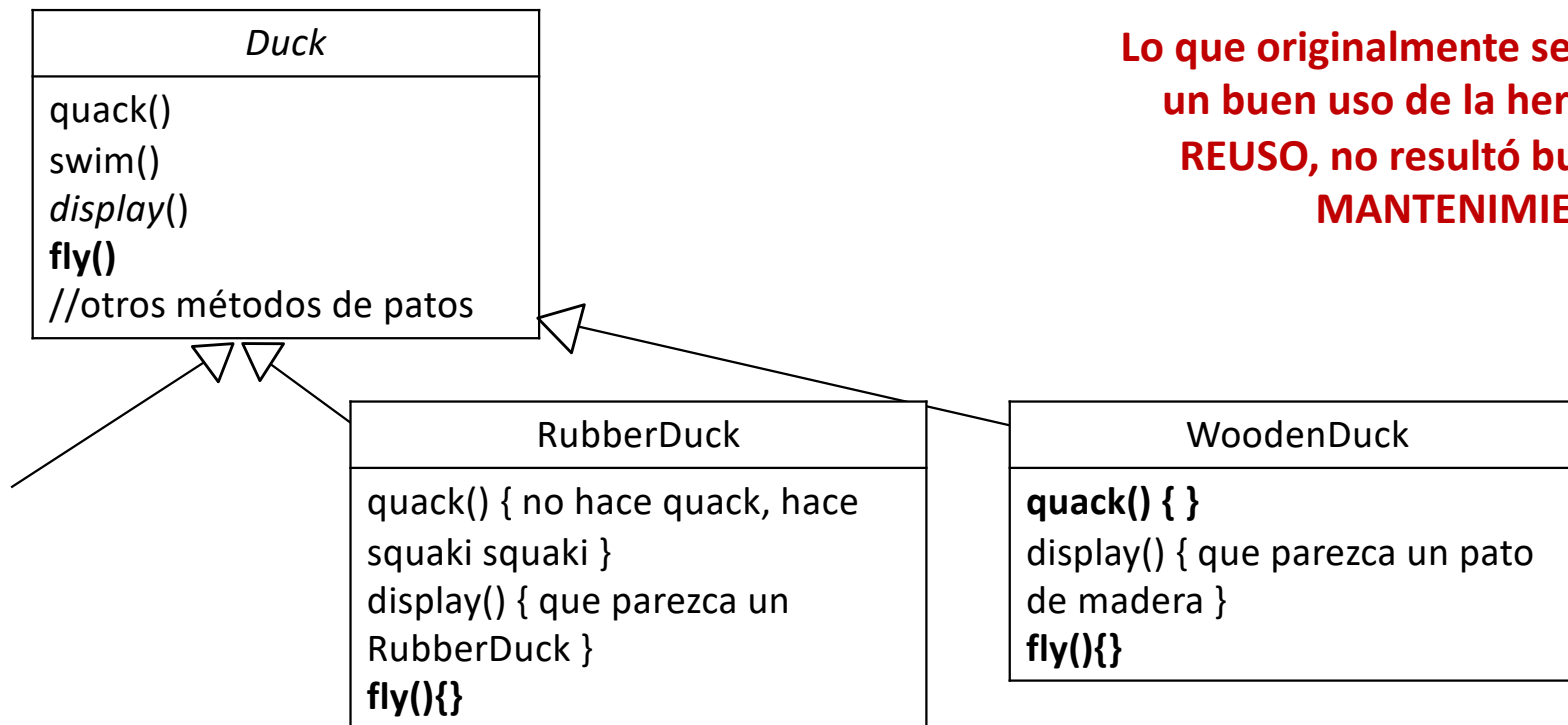
Vamos a arrancar con un ejemplo...

Pato, pato, ñato

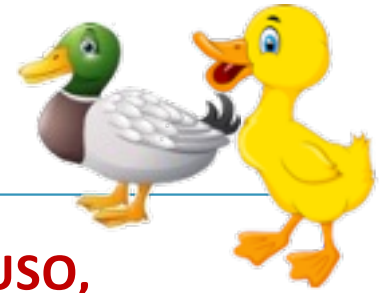


- Ahora queremos que los patos vuelen
- Agregamos el método en la clase pato

Lo que originalmente se pensó que era
un buen uso de la herencia para el
REUSO, no resultó bueno para el
MANTENIMIENTO



Vamos a arrancar con un ejemplo...



Pato, pato, ñato

Lo que originalmente se pensó que era un buen uso de la herencia para el REUSO, no resultó bueno para el MANTENIMIENTO

- Cuáles de las siguientes son desventajas de haber utilizado herencia para proveer comportamiento a los patos?

Hay código duplicado en las subclases

Los cambios de comportamiento en ejecución pueden ser complicados

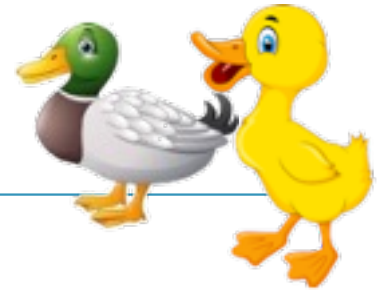
No se puede agregar más funcionalidad a los patos

Difícil conocer el comportamiento de todos los patos

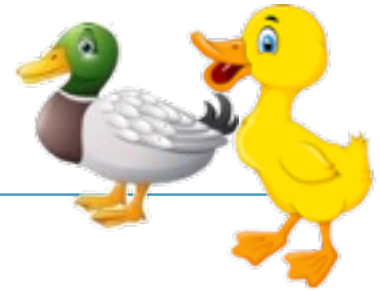
Los patos no pueden volar y hacer quack al mismo tiempo

Los cambios realizados no-intencionalmente afectan a otros patos

Qué salió mal? Cómo podemos mejorarlo?

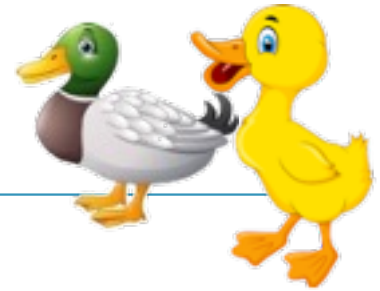


Uso de interfaces

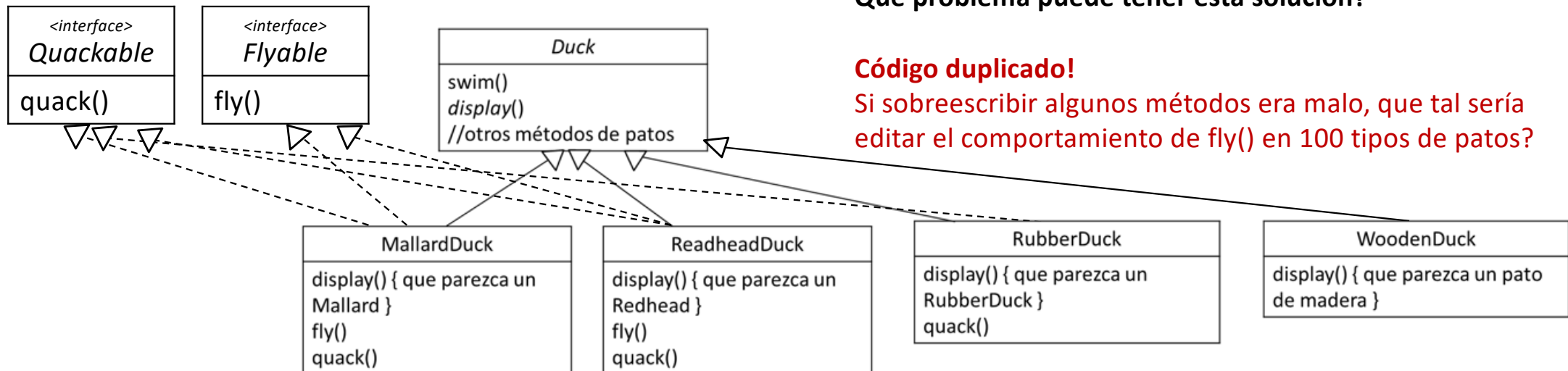


- La herencia puede no ser la mejor opción, más si hay que hacer modificaciones frecuentes al diseño
 - Modificaciones que a menudo no se sabe cuáles podrán ser...
- **Qué pasa si se agrega una interface (Java)?**
 - Se saca fly() de la clase Duck y se hace una interfaz Flyable que tenga el método deseado
 - De esta forma, solo los patos que se quiere que vuelen implementan ese método

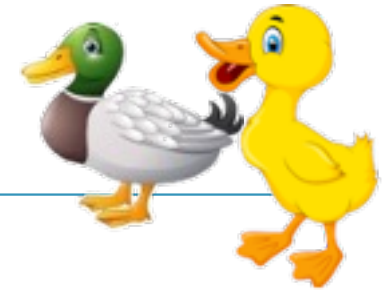
Pato, pato, ñato ... Interfaces



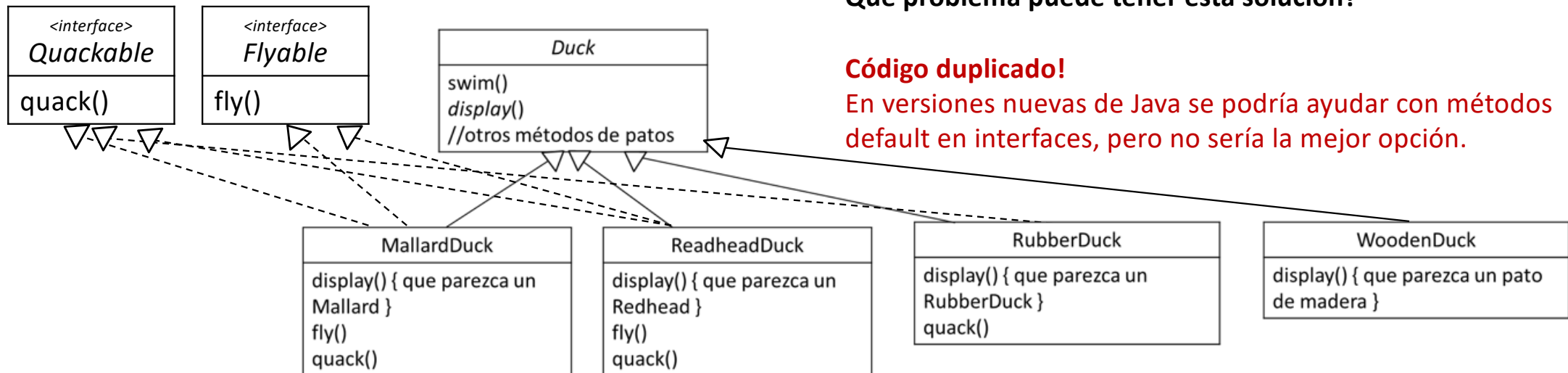
- La herencia puede no ser la mejor opción, más si hay que hacer modificaciones frecuentes al diseño
 - Modificaciones que a menudo no se sabe cuáles podrán ser...
- **Qué pasa si se agrega una interface (Java)?**
 - Se saca fly() de la clase Duck y se hace una interfaz Flyable que tenga el método deseado
 - De esta forma, solo los patos que se quiere que vuelen implementan ese método



Pato, pato, ñato ... Interfaces



- La herencia puede no ser la mejor opción, más si hay que hacer modificaciones frecuentes al diseño
 - Modificaciones que a menudo no se sabe cuáles podrán ser...
- **Qué pasa si se agrega una interface (Java)?**
 - Se saca fly() de la clase Duck y se hace una interfaz Flyable que tenga el método deseado
 - De esta forma, solo los patos que se quiere que vuelen implementan ese método

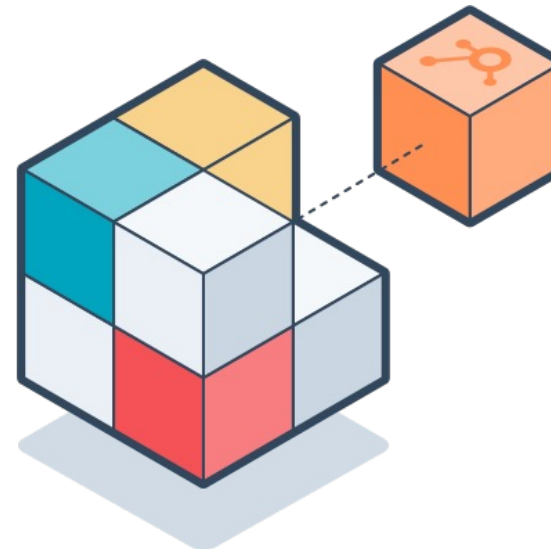


Cambios, cambios y más cambios

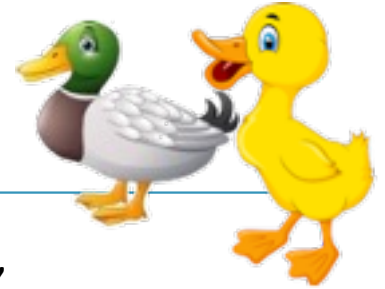
No importa qué se esté haciendo, en qué lenguaje de programación se esté programando, siempre van a existir...

CAMBIOS

Esto lleva a pensar en la **modificabilidad y modularidad** de un sistema de software, en lo posible, desde etapas tempranas



Cambios, cambios y más cambios



No importa qué se esté haciendo, en qué lenguaje de programación se esté programando, siempre van a existir...

CAMBIOS

Esto lleva a pensar en la **modificabilidad y modularidad** de un sistema de software, en lo posible, desde etapas tempranas

- Herencia no funcionó bien dado que el comportamiento no era adecuado para todos los patos
- Las interfaces, en general, complican el reuso de código, con lo que un cambio potencialmente genera tener que buscar dónde hacer esos cambios.

Identificar los aspectos que puedan variar y separarlos de lo que no vaya a cambiar

Encapsular esos aspectos de forma que luego se los pueda modificar o extender sin afectar aquello que no cambia

Patrones everywhere

Strategy

Decorator

Factory

Singleton

Adapter

Command

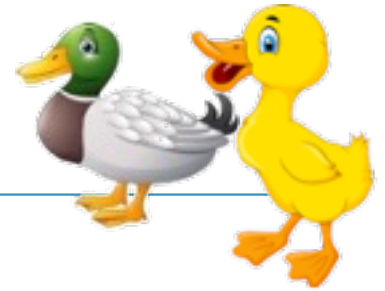
Visitor

Template

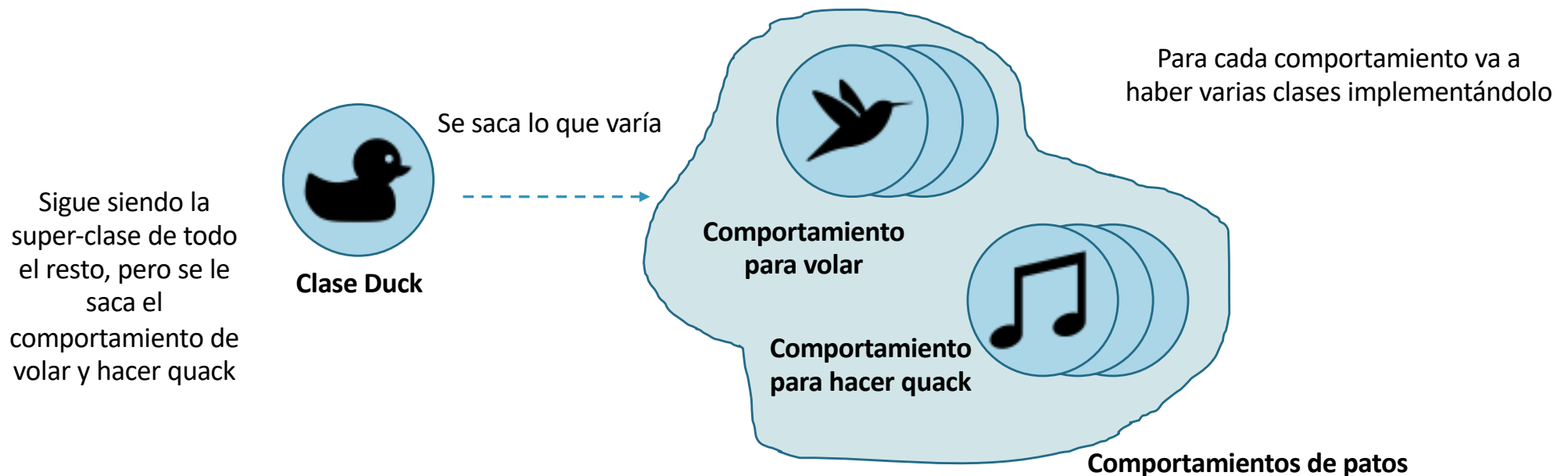


Pato, pato, ñato

Cambios, cambios y más cambios

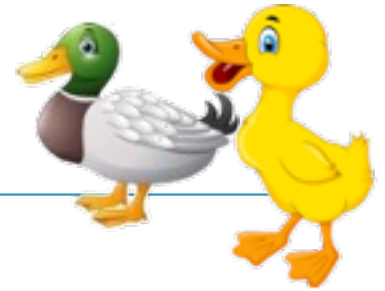


- Por dónde empezar?
- La clase Duck parece que va “bien”, sacando los problemas con fly() y quack().
- No hay otras partes que parezca que vayan a cambiar frecuentemente.



Pato, pato, ñato

Diseñando comportamientos



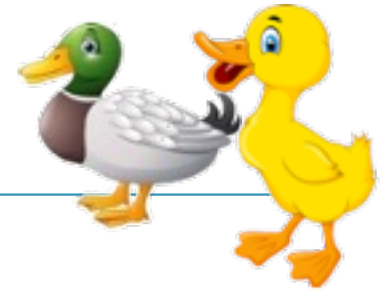
- Cómo se va a diseñar el conjunto de clases que implementan los comportamientos que se separaron?
- Se quiere que quede **flexible**
- Se quiere poder asignar esos comportamientos a las instancias de Duck
 - Por ejemplo, poder inicializar una instancia de RubberDuck con un tipo especial de comportamiento para volar que no haga nada
- Se quiere que ese comportamiento se pueda modificar de forma dinámica
 - Es decir, que se pueda cambiar en tiempo de ejecución

Programar para interfaces, no para implementaciones!

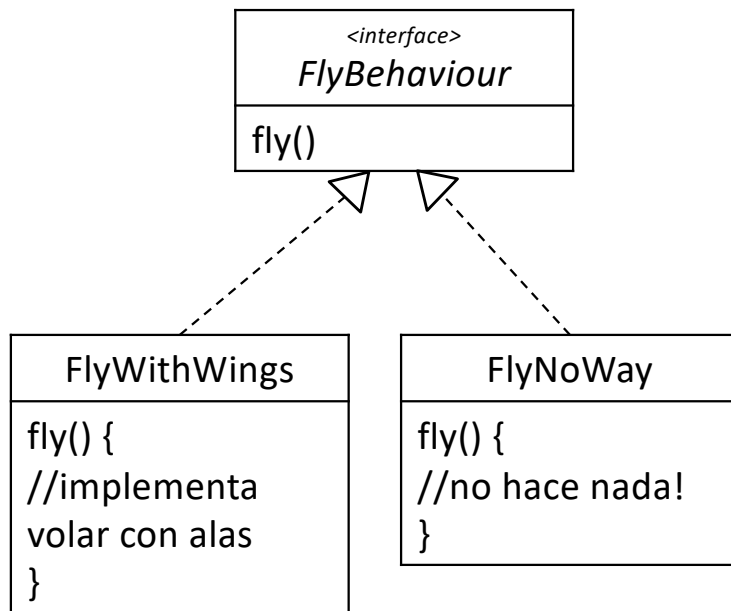
No atarse a una implementación específica!

Pato, pato, ñato

Diseñando comportamientos



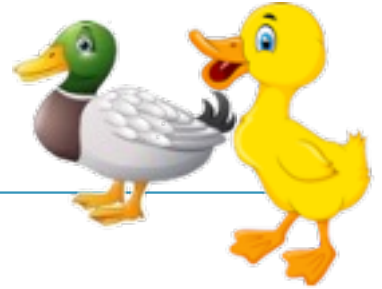
Programar para interfaces, no para implementaciones!



- Se va a usar una interface para representar el comportamiento y cada implementación implementará esa interfaz
- Las instancias de Duck NO implementarán estos comportamientos, sino que serán estas nuevas clases las que lo harán.
- Esto es diferente a lo anterior donde el comportamiento estaba o en la superclase o en alguna de las implementaciones.
- En ese caso, se está atado a una implementación, la cual no se podía cambiar (sin cambiar el código de la clase)
- Con esta nueva implementación, las subclases de Duck utilizarán un comportamiento representado por una *interface*, de forma que la *implementación* del comportamiento no estará fijo en las subclases de Duck.

Pato, pato, ñato

Diseñando comportamientos



Programar para interfaces → Programar para un super-tipo

- Aprovechar el polimorfismo!
- Se programa para un super-tipo para evitar que el objeto en tiempo de ejecución no se encuentre realmente definido en compilación
- Los objetos asignados a dichas variables pueden ser de cualquiera de las implementaciones concretas del super tipo
- La clase que las declara no necesita conocer los tipos específicos

Programar para una implementación

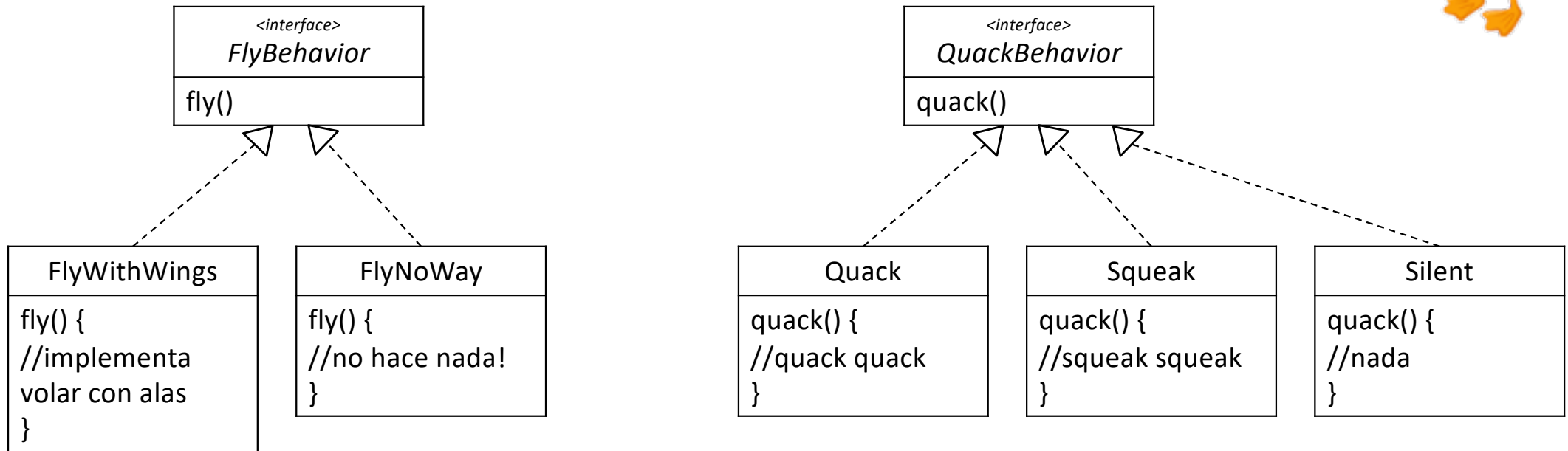
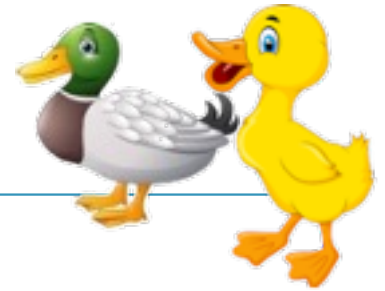
```
MallardDuck d = new MallardDuck();  
d.display();
```

Programar para un super-tipo

```
Duck d = new MallardDuck();  
d.display();
```

Pato, pato, ñato

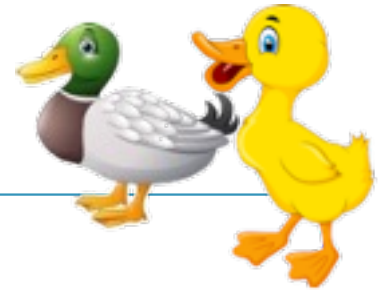
Diseñando comportamientos



- Con este diseño, otros tipos de objetos puede reutilizar el comportamiento de fly() y quack(), ya que no están más escondidos en la clases Duck
- Se puede agregar nuevo comportamiento sin modificar las clases de comportamiento ya existentes o alguna de las clases de Duck

Pato, pato, ñato

Diseñando comportamientos



- Duck ahora delegará sus comportamientos a las otras clases, con lo que vamos a modificar la clase

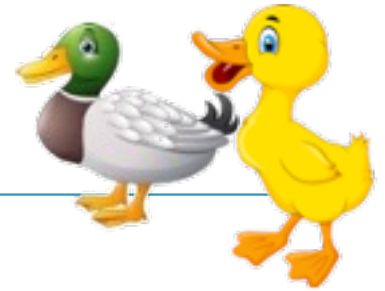
1 Se agregan variables de instancia

- Cada instancia tendrá estas variables polimórficas para referenciar el tipo de comportamiento que le corresponda en ejecución
- Se eliminan los métodos `fly()` y `quack()` dado que ahora los implementan las otras clases y se agregan métodos para invocar ese comportamiento

<i>Duck</i>
<code>flyBehavior : FlyBehavior</code> <code>quackBehavior : QuackBehavior</code>
<code>performQuack()</code> <code>swim()</code> <i><code>display()</code></i> <code>performFly()</code> <i>//otros métodos de patos</i>

Pato, pato, ñato

Diseñando comportamientos



- Duck ahora delegará sus comportamientos a las otras clases, con lo que vamos a modificar la clase

2 Se implementa performQuack()

```
public class Duck{  
    QuakBehavior quackBehavior;  
    // el resto...  
  
    public void performQuack(){  
        quackBehavior.quack();  
    }  
}
```

→ Cada Duck tiene una referencia a algo que implementa la interface QuackBehavior

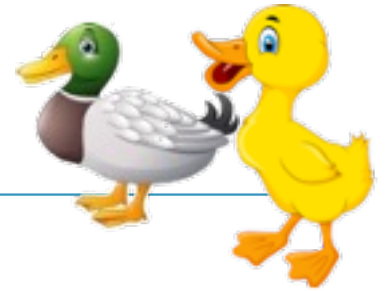
→ En lugar de implementar el comportamiento, lo que hace es delegar el comportamiento al objeto referenciado por quackBehavior

→ No nos importa qué tipo de objeto es, sino qué sepa como hacer quack!

Duck
flyBehaviour : FlyBehaviour quackBehaviour : QuackBehaviour
performQuack() swim() display() performFly() //otros métodos de patos

Pato, pato, ñato

Diseñando comportamientos



- Duck ahora delegará sus comportamientos a las otras clases, con lo que vamos a modificar la clase

3 Cómo se setean las variables?

Vamos a MallardDuck

```
public class MallardDuck{  
    public void MallardDuck(){  
        quackBehaviour = new Quack();  
        flyBehaviour = new FlyWithWings();  
    }  
}
```

Se va a delegar los comportamientos a estas clases

Variables heredadas de Duck

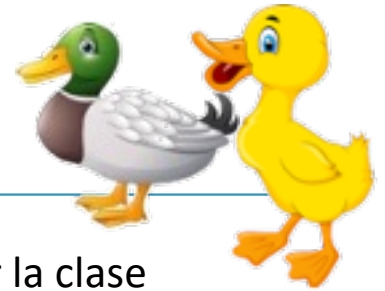
Duck
flyBehaviour : FlyBehaviour quackBehaviour : QuackBehaviour
performQuack() swim() display() performFly() //otros métodos de patos

La idea era no atarse a una implementación concreta... y qué estamos haciendo?

- Se está seteando el comportamiento en la inicialización de la instancia
- Es flexible, pero no tanto
- Podríamos aprovechar las interfaces para usar el polimorfismo y permitir cambios de forma dinámica

Pato, pato, ñato

Diseñando comportamientos



- Duck ahora delegará sus comportamientos a las otras clases, con lo cuál se va modificar la clase

4 Cómo se setea el comportamiento de forma dinámica?

Se agregan 2 nuevos métodos a la clase Duck

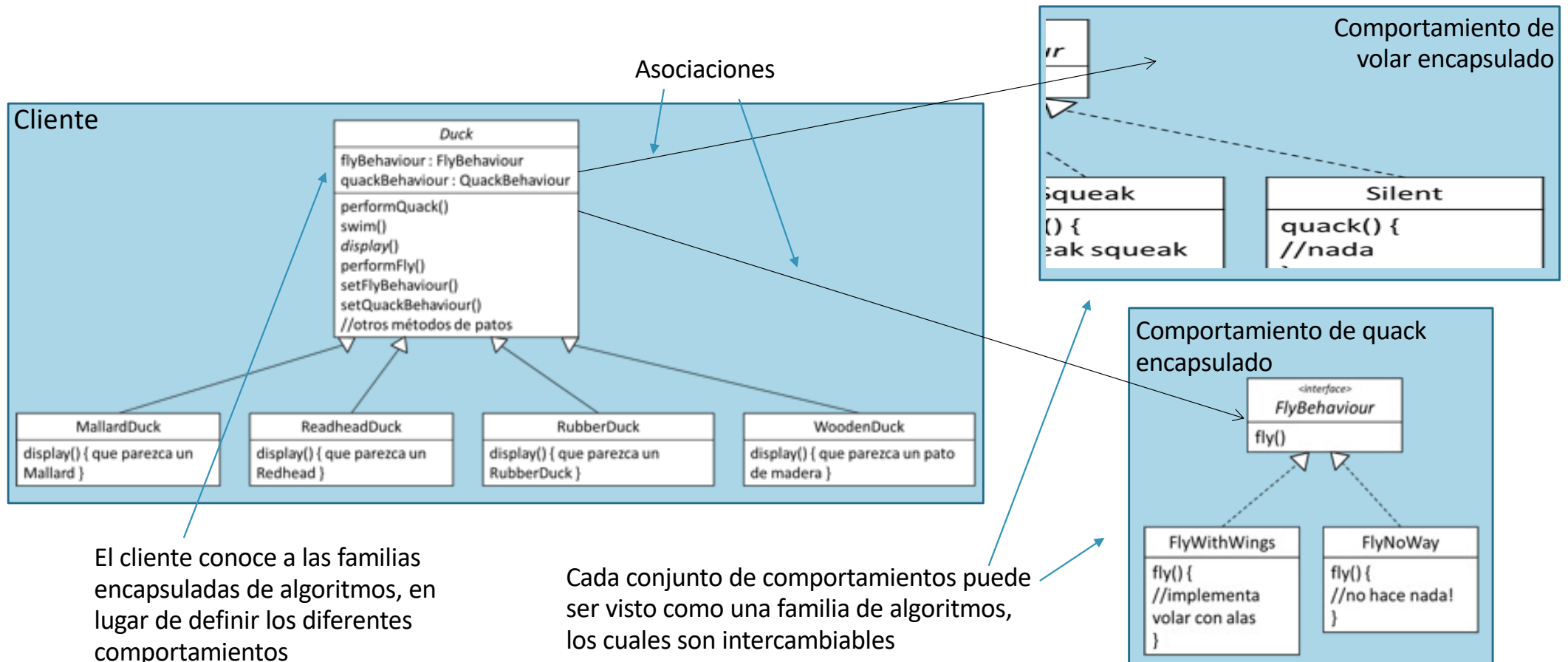
```
public void setFlyBehaviour(FlyBehavior fb){  
    flyBehavior = fb;  
}  
  
public void setQuackBehaviour(QuackBehavior qb){  
    quackBehavior = qb;  
}
```

<i>Duck</i>
flyBehavior : FlyBehavior quackBehavior : QuackBehavior
performQuack() swim() display() performFly() setFlyBehavior() setQuackBehavior() //otros métodos de patos

Ahora, se van a poder invocar estos métodos cuando se quiera, y cambiar los comportamientos en tiempo de ejecución

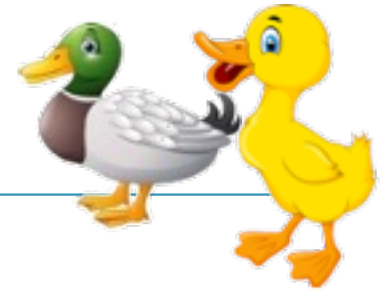
Pato, pato, ñato

Integrando comportamientos



Pato, pato, ñato

Integrando comportamientos



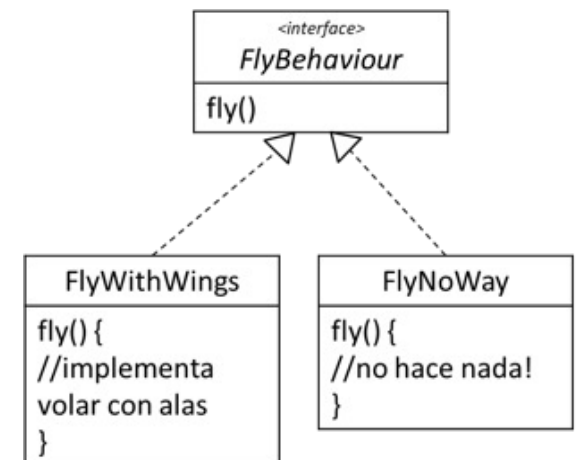
Favorecer la asociación/composición sobre la herencia

Es preferible el TIENE-UN antes que el ES-UN

- Usar asociaciones da mucha más flexibilidad
- No solo permite encapsular comportamiento en una familia o conjunto de clases, sino que además facilita los cambios en tiempo de ejecución

Patrón: Strategy

Define una familia de algoritmos, los encapsula y permite que se usen de forma intercambiable. Permite que los algoritmos varíen de forma independiente a los clientes que los usan



Patrón Strategy (recap)

Strategy

- Define una familia de algoritmos, los encapsula y los hace intercambiables
- Permite que el algoritmo varíe independientemente de los clientes que los usan

Principios OO

- Encapsular lo que varía
- Favorecer la composición por sobre la herencia
- Programar para interfaces, no para implementaciones

- Diseños buenos OO son reusables, extensibles y “fáciles” de mantener
- Los patrones ofrecen alternativas para diseñar sistemas “de calidad”
- Los patrones no proveen código, sino que son soluciones generales para problemas de diseño
- Los patrones no se inventan, se descubren
- La mayoría de los patrones se abocan a cuestiones relacionadas al cambio
- La mayoría de los patrones intentan ver qué es lo que cambia en el sistema para encapsularlo
- Los patrones proveen un lenguaje compartido

Preguntas



Patrones everywhere

Strategy

Decorator

Factory

Singleton

Adapter

Command

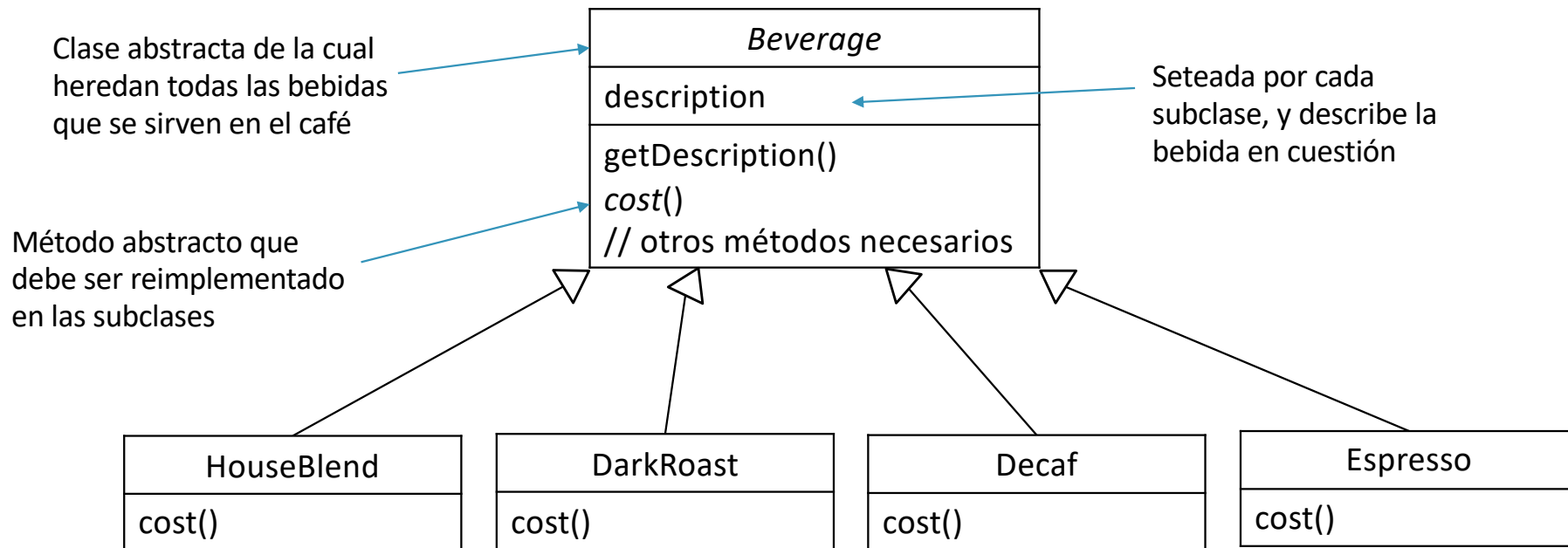
Visitor

Template



Decorator

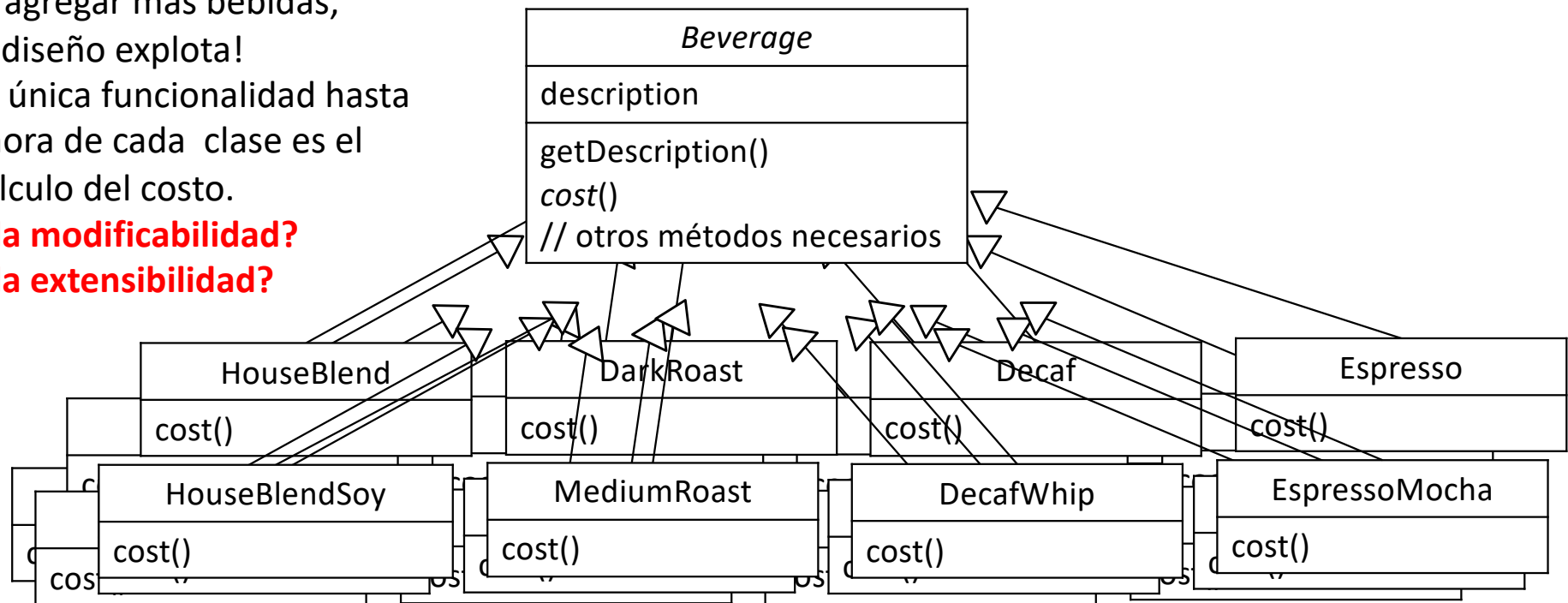
- Un café tiene la fama de agregar constantemente bebidas nuevas a su carta
- Su primer diseño de clases era:



- Además, se pueden combinar ingredientes, los cuales afectan el precio de la bebida

Decorator

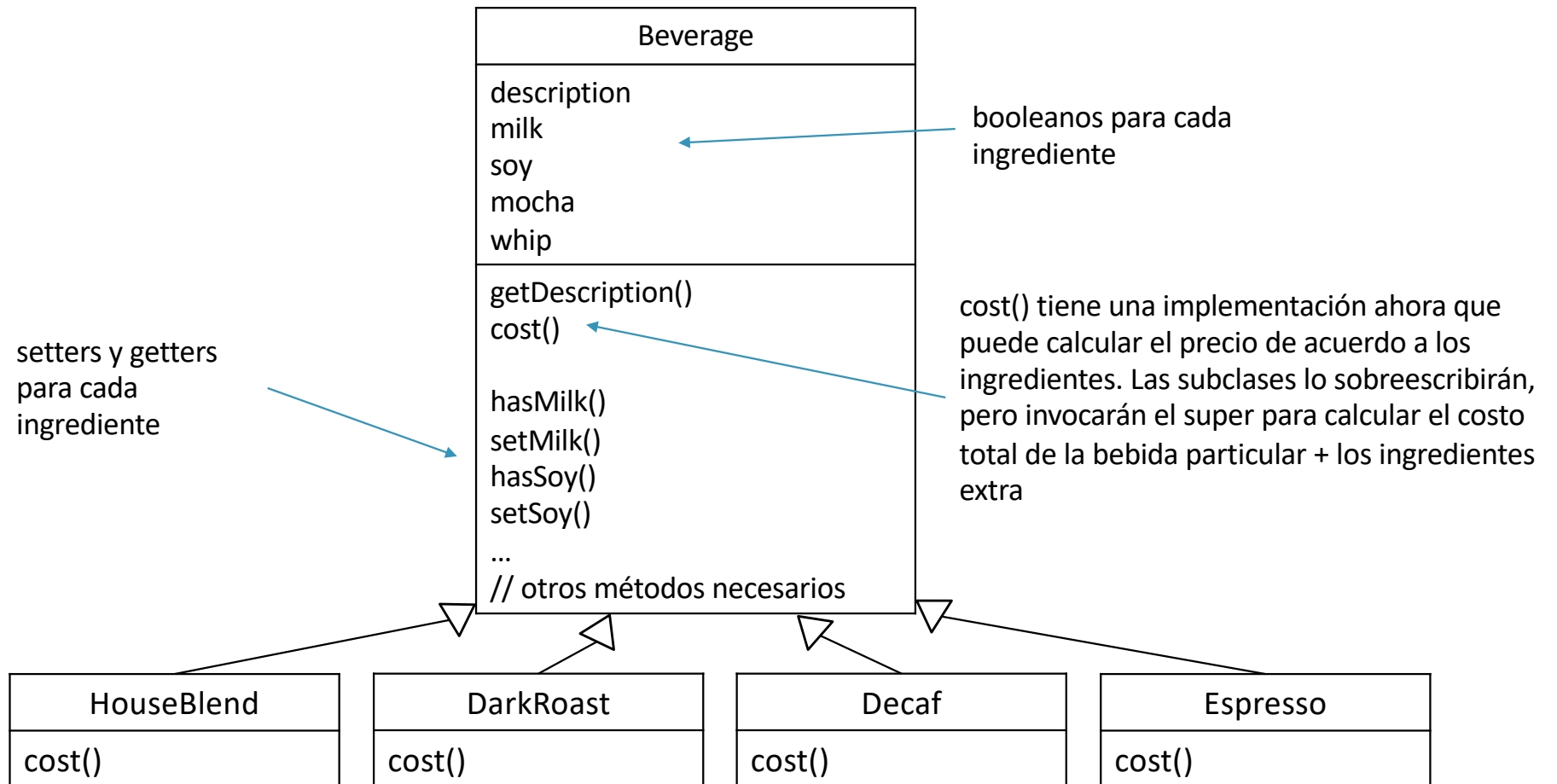
- Un café tiene la fama de agregar constantemente bebidas nuevas a su carta
- Al agregar más bebidas, el diseño explota!
- La única funcionalidad hasta ahora de cada clase es el cálculo del costo.
- **Y la modificabilidad?**
- **Y la extensibilidad?**



- Qué pasa cuando el precio de un ingrediente sube? O si se quiere agregar un nuevo ingrediente? No escala!

Decorator

Primera aproximación



Decorator

Primera aproximación

- Cómo quedarían implementados los siguientes métodos?

```
public class Beverage{  
    // nuevas variables de costo  
  
    public double cost(){  
        float cost = 0;  
        if (hasMilk())  
            cost += milkCost;  
        if (hasSoy())  
            cost += soyCost;  
        ...  
        return cost;  
    }  
    ...  
}
```

```
public class DarkRoast extends Beverage{  
  
    public DarkRoast(){  
        description = "Dark Roast Coffee"  
    }  
  
    public double cost(){  
        return super.cost() + 1.99;  
    }  
    ...  
}
```

Decorator

Primera aproximación

- Qué pasa con las posibilidades de cambio?
- Qué otros aspectos pueden cambiar que puedan generar problemas?
 - Cambios en los precios de los ingredientes
 - Nuevos ingredientes, implican nuevas variables, nuevos métodos
 - Nuevas bebidas que no requieran de todos los ingredientes, pero así y todo hereden todos los métodos
 - Qué pasa si un cliente quiere más de un ingrediente?

Decorator

Open, close



Las clases deben estar abiertas para extensión, pero cerradas para la modificación

Se puede extender las clases con el comportamiento que se quiera. Extender!

*Siempre y cuando no se modifique lo que ya está!
El código que ya se tiene, no se toca!*

- El objetivo es que las clases puedan ser fácilmente extensibles para agregar nuevo comportamiento sin modificar lo que ya está
- Qué se logra con esto?
 - Diseños flexibles y tolerantes a los cambios
- Pero, NO hay que aplicarlo en todo!
 - Puede ser costoso y dar lugar a código complejo y difícil de comprender

Decorator

Open, close

- Hacer una clase por cada posible combinación de bebida + ingrediente, no funciona.
- Hacer una clase para la bebida base y agregarle ingredientes, nos deja un diseño poco flexible
- Tener todos los ingredientes en la super-clase hace que todas las subclases los hereden, aún cuando la combinación no sea posible
- Para esto, está el patrón **Decorator**.
 - Se va a tener una bebida base y la vamos a “decorar” en tiempo de ejecución
- Supongamos que un cliente quiere un DarkRoast con Mocha y crema Whip.

1 Se comienza con el objeto de DarkRoast



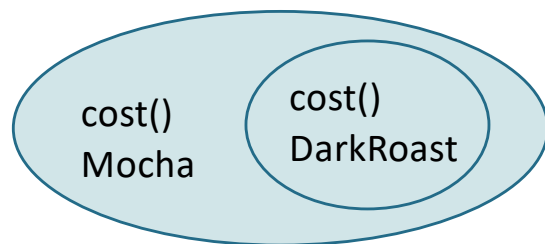
Hereda de Beverage y
tiene el método cost()

Decorator

Open, close

- Hacer una clase por cada posible combinación de bebida + ingrediente, no funciona.
- Hacer una clase para la bebida base y agregarle ingredientes, nos deja un diseño poco flexible
- Tener todos los ingredientes en la super-clase hace que todas las subclases los hereden, aún cuando la combinación no sea posible
- Para esto, está el patrón **Decorator**.
 - Se va a tener una bebida base y la vamos a “decorar” en tiempo de ejecución
- Supongamos que un cliente quiere un DarkRoast con Mocha y crema Whip.

2 Hay que agregar el Mocha. Se crea el objeto y lo usamos para “envolver” al DarkRoast



Mocha es un decorator.

Mocha a su vez es una bebida!

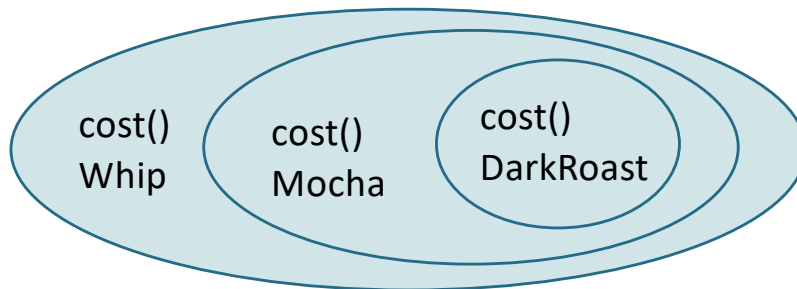
Mocha sabe también calcular su costo! Gracias al polimorfismo. Se puede tratar a cualquier Beverage envuelta en Mocha como una Beverage, porque lo es

Decorator

Open, close

- Hacer una clase por cada posible combinación de bebida + ingrediente, no funciona.
- Hacer una clase para la bebida base y agregarle ingredientes, nos deja un diseño poco flexible
- Tener todos los ingredientes en la super-clase hace que todas las subclases los hereden, aún cuando la combinación no sea posible
- Para esto, está el patrón **Decorator**.
 - Se va a tener una bebida base y la vamos a “decorar” en tiempo de ejecución
- Supongamos que un cliente quiere un DarkRoast con Mocha y crema Whip.

3 Hay que agregar el Whip



Al igual que Mocha, Whip es un decorator

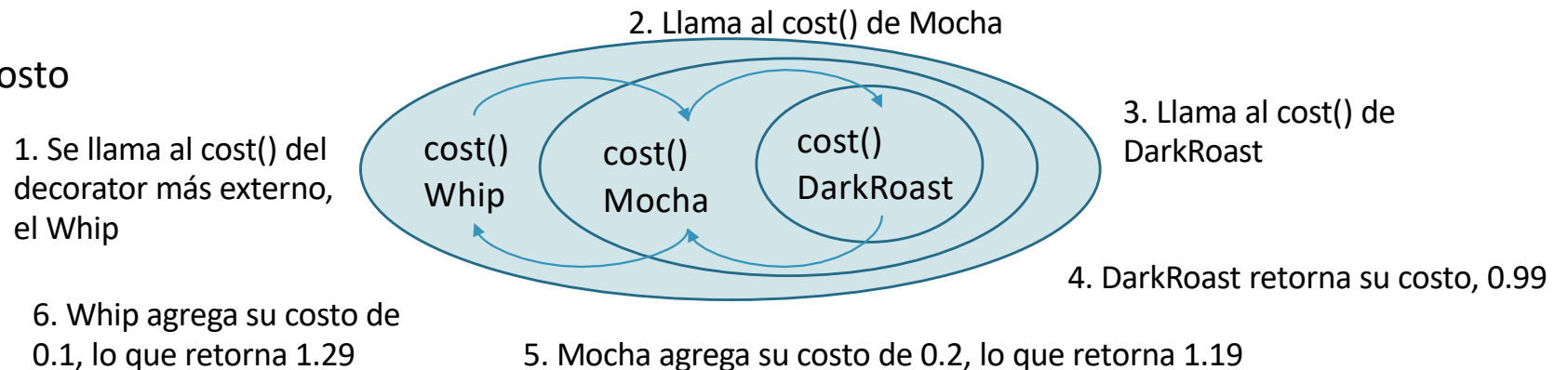
Decorator

Open, close

- Hacer una clase por cada posible combinación de bebida + ingrediente, no funciona.
- Hacer una clase para la bebida base y agregarle ingredientes, nos deja un diseño poco flexible
- Tener todos los ingredientes en la super-clase hace que todas las subclases los hereden, aún cuando la combinación no sea posible
- Para esto, está el patrón **Decorator**.
 - Se va a tener una bebida base y la vamos a “decorar” en tiempo de ejecución
- Supongamos que un cliente quiere un DarkRoast con Mocha y crema Whip.

4

Se calcula el costo



Decorator

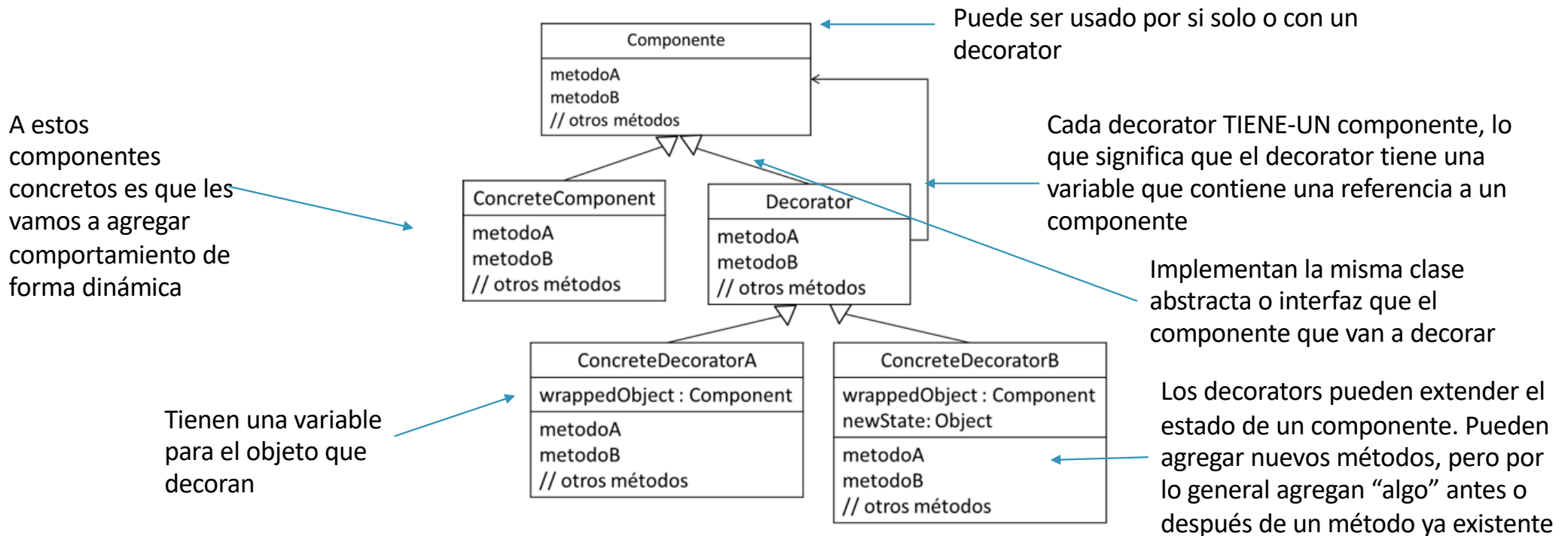
Open, close

- Hacer una clase por cada posible combinación de bebida + ingrediente, no funciona.
- Hacer una clase para la bebida base y agregarle ingredientes, nos deja un diseño poco flexible
- Tener todos los ingredientes en la super-clase hace que todas las subclases los hereden, aún cuando la combinación no sea posible
- Para esto, está el patrón **Decorator**.
 - Se va a tener una bebida base y la vamos a “decorar” en tiempo de ejecución
- Supongamos que un cliente quiere un DarkRoast con Mocha y crema Whip.
- **Qué sabemos hasta ahora?**
 - *Los Decorators tienen el mismo supertipo que los objetos que decoran*
 - *Dado que tienen el mismo tipo, se los puede ir pasando y usar en lugar del objeto original*
 - *El Decorator agrega su propio comportamiento antes y/o después de realizar alguna delegación al objeto que envuelve*
 - *Los objetos pueden ser decorados en cualquier momento de forma dinámica*

Decorator

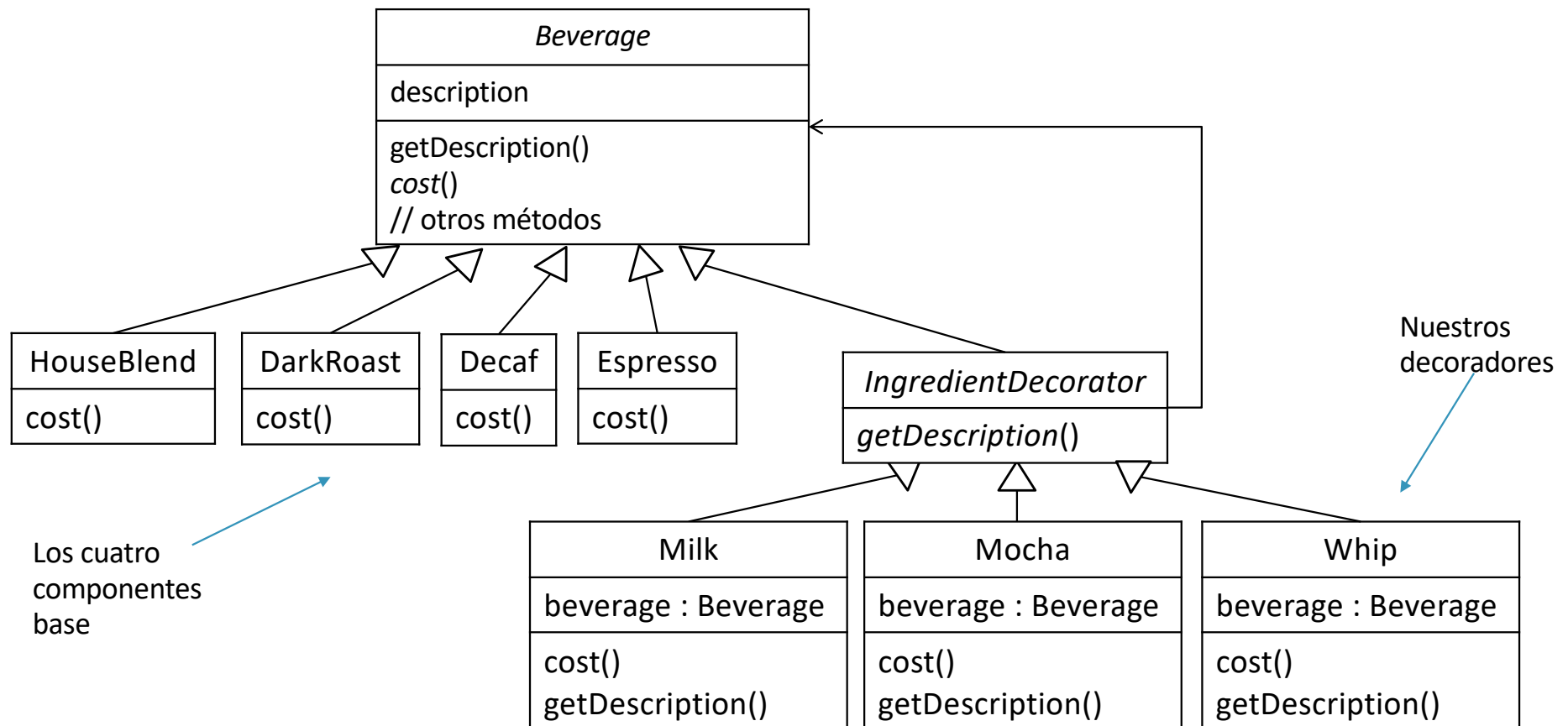
Qué es?

Agrega responsabilidades adicionales a un objeto de forma dinámica
Proveen una alternativa flexible a la creación de subclasses para el agregado de comportamiento



Decorator

Y cómo quedan los cafés?



Decorator

Y cómo quedan los cafés?

- Veamos código

```
public abstract class Beverage{  
    String description = "Unknown Beverage";  
  
    public String getDescription(){  
        return description;  
    }  
  
    public abstract double cost();  
}
```

← Clase abstracta con 2 métodos

← Tiene una descripción base, pero se va a reimplementar

```
public abstract class IngredientDecorator extends Beverage{  
    public abstract String getDescription();  
}
```

← Puede ser usada de forma intercambiable con Beverage

Decorator

Y cómo quedan los cafés?

```
public Espresso class Beverage{
```

Se extiende la
clase Beverage

```
    public Espresso(){  
        description = "Espresso";  
    }
```

Se configura la
description para
esta bebida

```
    public double cost(){  
        return 1.99;  
    }
```

Acá el costo es solo de la
bebida base, sin ningún
ingrediente extra

Así quedan todas
las bebidas base

```
public HouseBlend class Beverage{  
  
    public HouseBlend(){  
        description = "House Blend";  
    }  
  
    public double cost(){  
        return 1.99;  
    }  
}
```

Decorator

Y cómo quedan los cafés?

```
public class Mocha extends IngredientDecorator{  
    Beverage beverage;
```

Se extiende IngredientDecorator que a su vez extiende a la clase Beverage

```
    public Mocha(Beverage beverage){  
        this.beverage = beverage;  
    }
```

Se instancia a Mocha con una referencia a una beverage

```
    public String getDescription(){  
        return beverage.getDescription() + ", Mocha";  
    }
```

Se quiera que la descripción no solo sea la de la bebida que se wrappea sino que incluya el extra que se le está agregando

```
    public double cost(){  
        return beverage.cost() + .20;  
    }  
}
```

Para calcular el costo, se obtiene el costo de la bebida a la que se está wrappeando y se le suma el costo particular del Mocha.

Así quedan todos los ingredientes

Decorator

Y cómo quedan los cafés?

- Para crear una bebida...

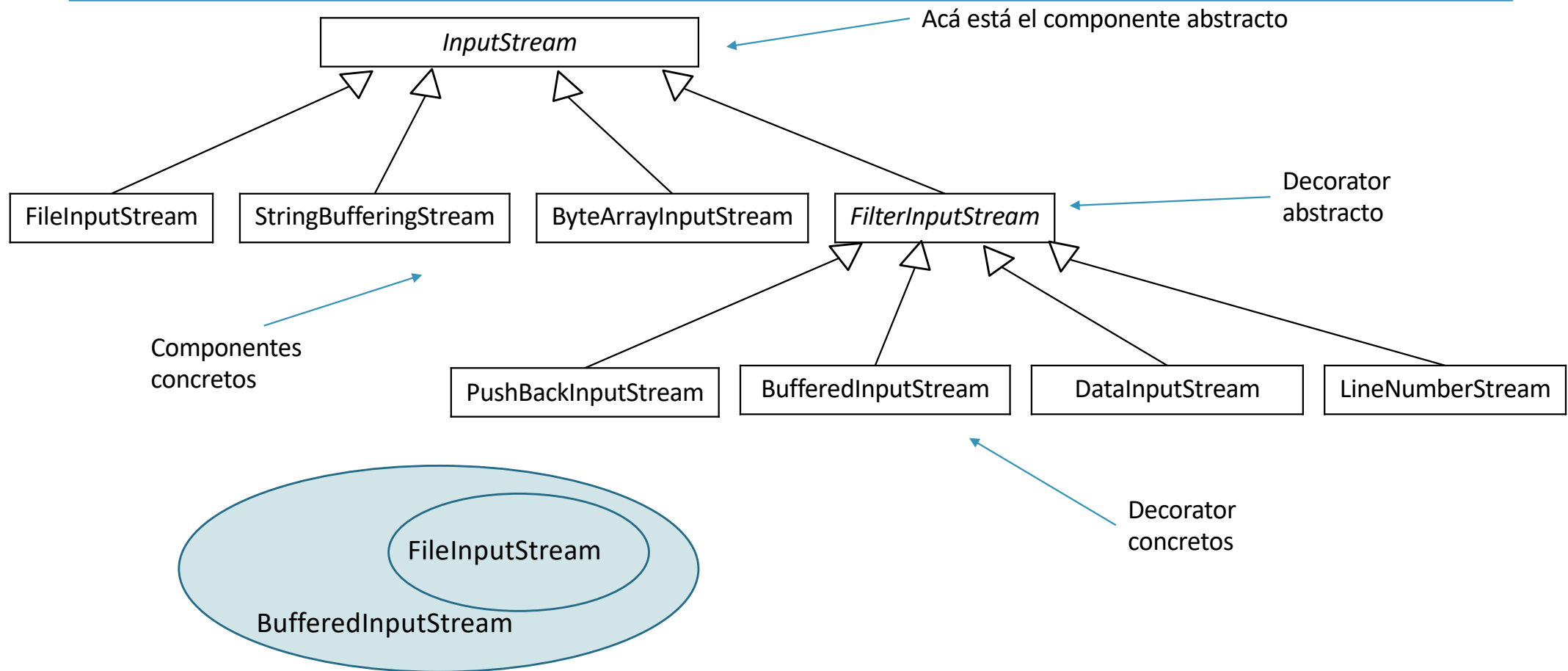
```
// Un dark roast con doble mocha y whip
```

```
Beverage bev1 = new DarkRoast();  
bev1 = new Mocha(bev1);  
bev1 = new Mocha(bev1);  
bev1 = new Whip(bev1);
```

```
// Un House Blend con Soy, Mocha y Whip
```

```
Beverage bev2 = new HouseBlend();  
bev2 = new Soy(bev2);  
bev2 = new Mocha(bev2);  
bev2 = new Whip(bev2);
```


Decorators en clases de Java (JDK)



Decorator (recap)

Decorator

- Agregar responsabilidades adicionales de forma dinámica a un objeto
- Proveen una alternativa flexible a la creación de subclases para el agregado de comportamiento

Principios OO

- Las clases deben estar abiertas para la extensión pero cerradas para la modificación

- La herencia es una forma de extensión que quizás no sea la mejor opción para lograr flexibilidad
- Composición/Asociación y delegación son otras formas de agregar comportamiento en tiempo de ejecución
- El Decorator provee una alternativa a la creación de subclases para el agregado de comportamiento
- El Decorator involucra un conjunto de clases que se usan para decorar componentes concretos
- Los Decorators son del mismo tipo que las clases que decoran.
- Los Decorators cambian el comportamiento de los componentes agregando nueva funcionalidad antes, después o en lugar de invocaciones a métodos del componente
- Se puede utilizar una cantidad arbitraria de Decorators
- Son usualmente “transparentes” para el (objeto) usuario.
- Puede resultar en muchos objetos pequeños
- Su sobre-uso puede derivar en programas más complejos

Patrones everywhere

Strategy

Decorator

Factory

Singleton

Adapter

Command

Visitor

Template



Factory

Creando objetos

- Se comentó de programar para una interfaz, y tratar de no atarse a una implementación concreta, ya que esto reduce la flexibilidad de la solución

```
Pizza pizza = new CheesePizza();
```

Interfaces para mantener la
implementación flexible

Se crea una instancia de
una clase concreta

- Si se tienen varias clases concretas, es algo así:

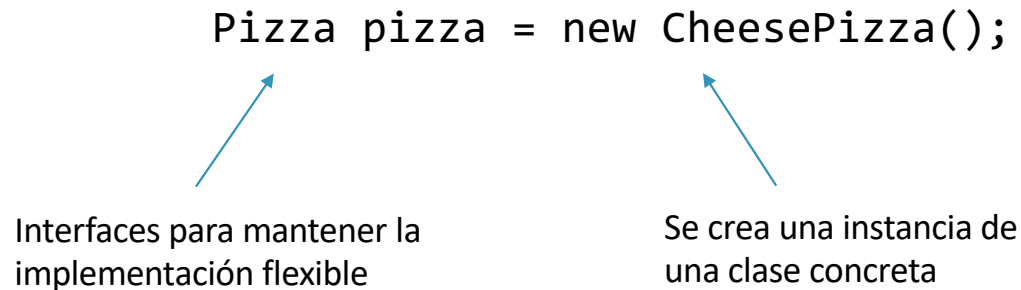
```
Pizza pizza;  
if (cheese)  
    pizza = new CheesePizza();  
else if (greek)  
    pizza = new GreekPizza();
```

Se tienen varias clases de pizzas
y no se sabe hasta la ejecución
cuál es la que se va a necesitar
instanciar

Factory

Creando objetos

- Se comentó de programar para una interfaz, y tratar de no atarse a una implementación concreta, ya que esto reduce la flexibilidad de la solución



- Usando interfaces se puede hacer adaptaciones, incluso a implementaciones que todavía no existen (polimorfismo)
- Si el código hace mucho uso de clases concretas, hay que hacer cambios con cada nueva clase que se agregue, con lo que no quedará “cerrado a modificaciones”

Factory – version 1

Pizzeria Panucci



- Hay que administrar una pizzeria

```
Pizza orderPizza(){  
    Pizza pizza = new Pizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

Por flexibilidad esto debiera ser una clase abstracta o interfaz, pero no se puede instanciar ninguna de esas

```
Pizza orderPizza(String type){  
    Pizza pizza;
```

Se necesita saber qué pizza hacer

```
    if(type.equals("cheese")  
        pizza = new CheesePizza();  
    else if(type.equals("greek")  
        pizza = new GreekPizza();  
    else if(type.equals("pepperoni")  
        pizza = new PepperoniPizza();
```

En base al tipo, se instancia la clase concreta correcta. Todas las pizzas concretas deben implementar Pizza

- Se agregan más pizzas...

```
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;
```

Una vez que se sabe qué pizza es, hay que hacerla. Cada pizza sabe como prepararse

```
}
```

Factory – version 1

Pizzeria Panucci



- Ahora, hay que agregar más pizzas!

```
Pizza orderPizza(String type){
    Pizza pizza;

    if(type.equals("cheese")
        pizza = new CheesePizza();
    else if(type.equals("greek")
        pizza = new GreekPizza();
    else if(type.equals("pepperoni")
        pizza = new PepperoniPizza();
    else if(type.equals("clam")
        pizza = new ClamPizza();
    else if(type.equals("veggie")
        pizza = new VeggiePizza();

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

Debería haber otra forma para poder encapsular la creación de los objetos...

El código queda cerrado para modificaciones. Cada vez que se cambien las pizzas que se ofrecen, hay que alterar este código

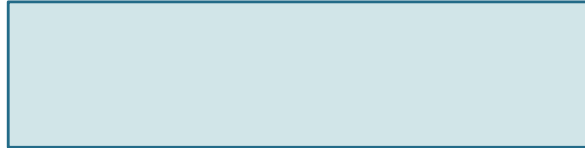
Esto es lo que se espera que **no cambie**. Si las implementaciones particulares, pero no la secuencia de pasos

Factory – version 1

Pizzeria Panucci



```
Pizza orderPizza(String type){  
    Pizza pizza;
```



```
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;
```

```
}
```

Se saca el código para la creación del objeto de método orderPizza(...)

```
if(type.equals("cheese")  
    pizza = new CheesePizza();  
else if(type.equals("greek")  
    pizza = new GreekPizza();  
else if(type.equals("pepperoni")  
    pizza = new PepperoniPizza();  
else if(type.equals("clam")  
    pizza = new ClamPizza();  
else if(type.equals("veggie")  
    pizza = new VeggiePizza();
```

Esto se llama patrón

Factory

Este código va a ir a una clase que solo va a preocuparse de cómo crear las pizzas. Si cualquier objeto necesita crear pizzas, deberá usar esta clase.

Factory – version 1

Pizzeria Panucci



```
public class PizzaStore {
```

```
    SimplePizzaFactory factory;  
  
    public PizzaStore(SimplePizzaFactory factory){  
        this.factory = factory;  
    }
```

Se da una referencia al
SimplePizzaFactory (el creador de pizzas)

```
    public Pizza orderPizza(String type){
```

```
        Pizza pizza = factory.createPizza(type);
```

```
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;
```

Se reemplaza la instanciación concreta de
las pizzas por una invocación a un
método

Esto también puede encontrarse con
métodos static

```
    }  
}
```

Factory – version 1

Pizzeria Panucci



```
public class SimplePizzaFactory {
```

← Su función es proveer pizzas a quienes las necesiten

```
    public Pizza createPizza(String type){
```

← Este es el método que se usará para instanciar los objetos

```
        Pizza pizza = null;
```

```
        if(type.equals("cheese"))
```

```
            pizza = new CheesePizza();
```

```
        else if(type.equals("greek"))
```

```
            pizza = new GreekPizza();
```

```
        else if(type.equals("pepperoni"))
```

```
            pizza = new PepperoniPizza();
```

```
        else if(type.equals("clam"))
```

```
            pizza = new ClamPizza();
```

```
        else if(type.equals("veggie"))
```

```
            pizza = new VeggiePizza();
```

← Este es el código se sacó de la clase Pizza

Sigue parametrizado como el método original

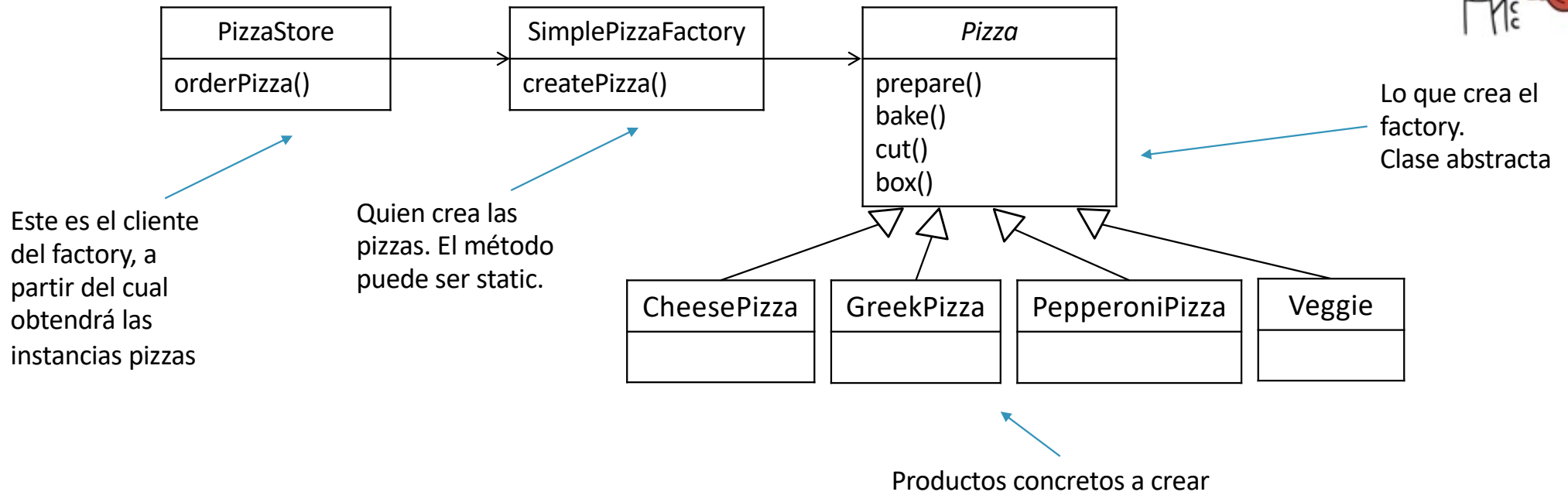
```
        return pizza;
```

```
    }
```

```
}
```

Factory – version 1

Pizzeria Panucci



Factory – version 2

Pizzeria Panucci ... Ahora con franquicias



- Se quieren agregar franquicias sobre las cuales poder controlar los productos ofrecidos

Todas las franquicias usan el mismo PizzaStore para que las pizzas sean creadas de la misma forma



PizzaStore



NYPizzaFactory



ChicagoPizzaFactory

Una franquicia quiere hacer pizza al estilo de New York

- Se crean factories nuevos que puedan hacer las pizzas para esos estilos

Se crea el factory para las pizzas de NY y se pasa al PizzaStore

```
NYPizzaFactory nyFactory = new NYPizzaFactory();  
PizzaStore nyStore = new PizzaStore(nyFactory);  
nyStore.orderPizza("veggie");
```

```
ChicagoPizzaFactory nyFactory = new ChicagoPizzaFactory();  
PizzaStore nyStore = new PizzaStore(nyFactory);  
nyStore.orderPizza("veggie");
```

- Esto da control sobre una parte de la fabricación, respecto a los tipos, pero no deja controlar exactamente ni la selección de ingredientes ni el proceso. Cómo hacer para que quede flexible?

Factory – version 2

Pizzeria Panucci: Framework



- Se modifica un poco el código:
 - createPizza() vuelve a PizzaStore como un método abstracto
 - Hay subclasses de PizzaStore para las regionales

PizzaStore ahora va a tener subclasses que van a decidir cómo se hace la pizza, pero que van a seguir utilizando los métodos de cut, box y demás de la clase padre

```
public abstract class PizzaStore{  
    public Pizza orderPizza(String type){  
        Pizza pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
  
        return pizza;  
    }  
  
    abstract Pizza createPizza(String type);  
}
```

Ahora es abstracta

createPizza vuelve a llamar a un método de PizzaStore en lugar del factory

Igual que antes

La llamada al factory va a estar acá

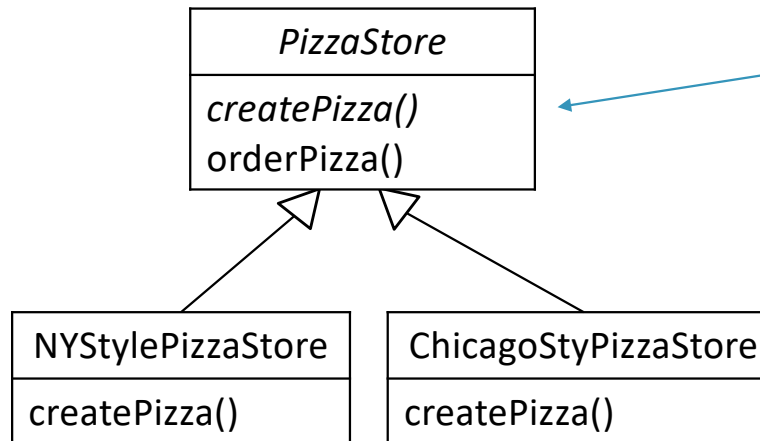
Factory – version 2

Pizzeria Panucci: Framework



- Se modifica un poco el Código:
 - createPizza() vuelve a PizzaStore como un método abstracto
 - Hay subclasses de PizzaStore para las regionales
- PizzaStore ahora va a tener subclasses que van a decidir cómo se hace la pizza, pero que van a seguir utilizando los métodos de cut, box y demás de la clase padre

```
if(type.equals("cheese")
    pizza = new NYStyleCheesePizza();
else if(type.equals("greek")
    pizza = new NYStyleGreekPizza();
else if(type.equals("pepperoni")
    pizza = new
NYStylePepperoniPizza();
else if(type.equals("clam")
    pizza = new NYStyleClamPizza();
else if(type.equals("veggie")
    pizza = new NYStyleVeggiePizza();
```



Cada subclase provee una implementación del createPizza(), y usan el orderPizza() del padre. Si se quiere realmente reforzar que todos los hijos usen el método del padre sin modificaciones, se puede declarar final

Es acá donde se crean las pizzas concretas. En el orderPizza de PizzaStore no se sabe qué se va a crear, sino simplemente los pasos comunes a todas

Factory – version 2

Pizzeria Panucci



- Volviendo al código de PizzaStore

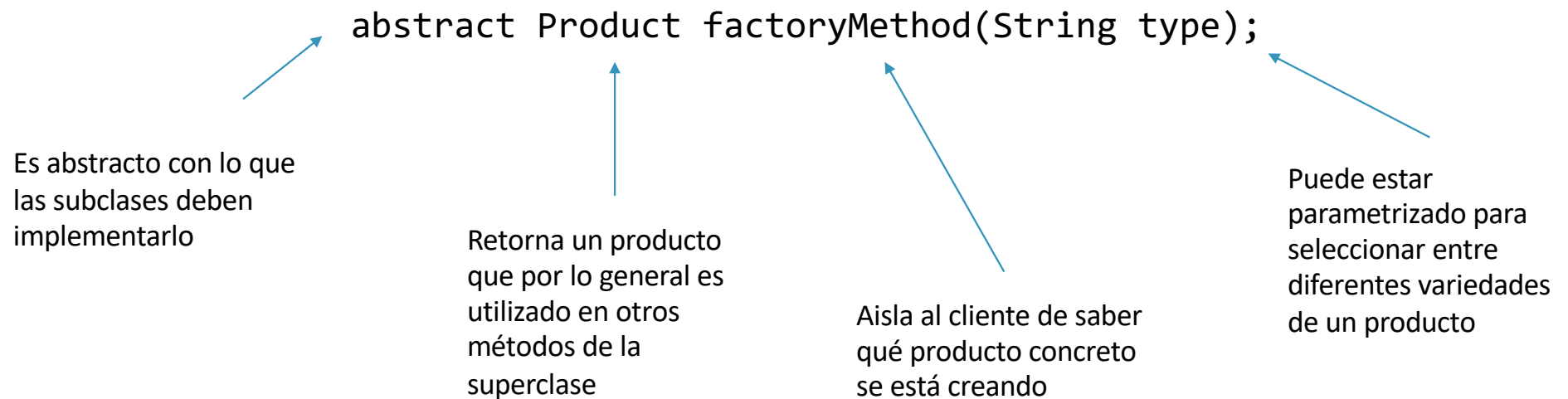
```
public abstract class PizzaStore{  
  
    public Pizza orderPizza(String type){  
        Pizza pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
  
        return pizza;  
    }  
  
    abstract Pizza createPizza(String type);  
}
```

Toda la responsabilidad para instanciar las pizzas fue movida a un método que funciona como un **Factory**

Esto se lo conoce como **Factory Method**

Factory Method

Un Factory Method se encarga de la creación de objetos y la encapsula en una subclase
Desacopla el Código del cliente en la superclase de la creación de los objetos en la subclase
Permite que las subclases “decidan” qué objetos crear



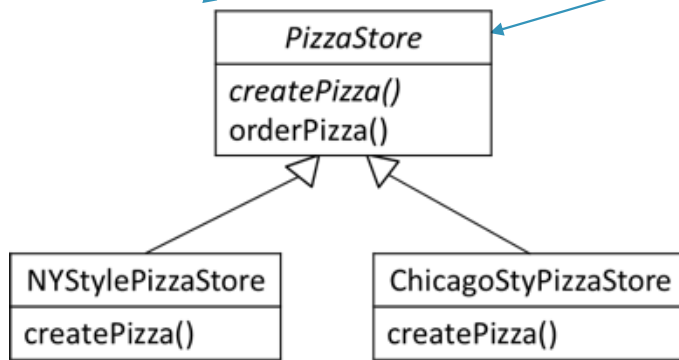
Factory Method

Estructuras de clases

Las clases creadoras

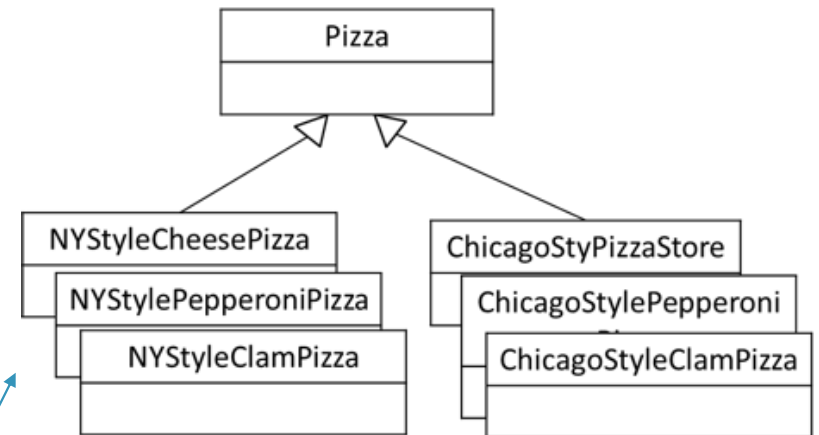
Clase abstracta. Define el abstract factory method que las subclases deben implementar

A menudo contiene código que depende de un producto abstracto, el cual es producido por una de las subclases. El creador no sabe que producto concreto fue creado.



El createPizza es el factory method. Producen los productos, de la forma que quieran

Las clases producto

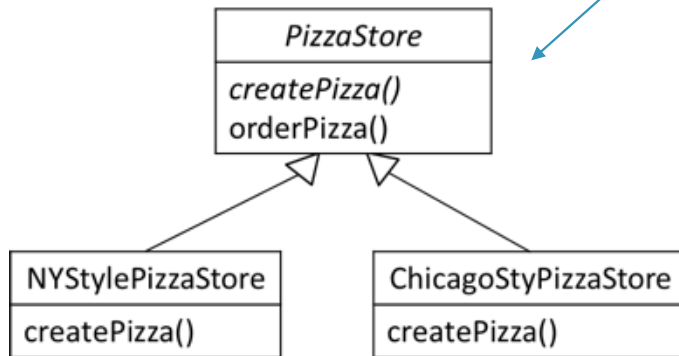


Los productos concretos, todos los tipos de pizzas que se ofrezcan

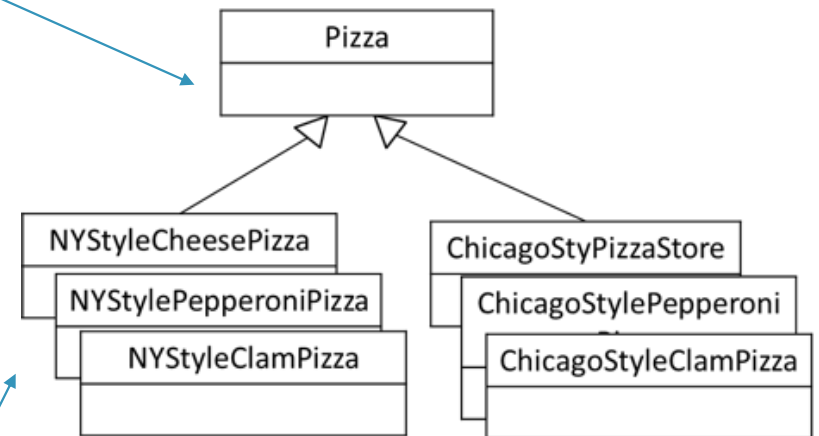
Factory Method

Estructuras de clases: Cómo se relacionan?

Las clases creadoras



Las clases producto

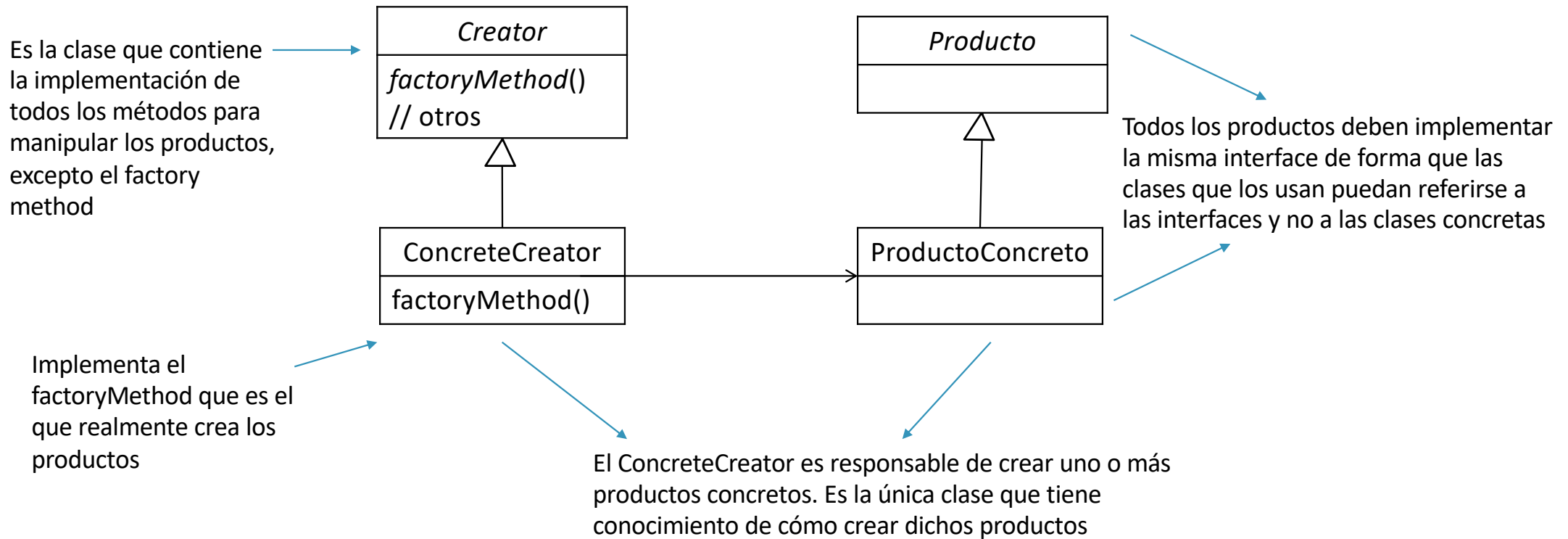


Las jerarquías son paralelas.
Ambas tienen clases
abstractas extendidas por
clases concretas

Encapsulan el conocimiento
para crear las pizzas de estilo
New York

Factory Method

Estructuras de clases: Cómo se relacionan?



Preguntas



Factory

Pedimos una pizza?



- Una cheese pizza estilo New York

1 Hace falta un NY Pizza Store

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

2 Ahora se puede tomar la orden

```
nyPizzaStore.orderPizza("cheese");
```

El método orderPizza es llamado sobre la instancia nyPizzaStore

3 El método orderPizza invoca al método createPizza

```
Pizza pizza = createPizza("cheese");
```

El factory method createPizza es implementado en la subclase, en este caso retornará una NY cheese pizza

4 Finalmente, tenemos la pizza, terminamos de prepararla

```
pizza.prepare();    pizza.bake();  
pizza.cut();        pizza.box();
```

Estos métodos son invocados sobre una pizza sin saber exactamente qué clase concreta es.

Factory

Pizzeria Panucci



```
public abstract class Pizza{  
    String name;  
    String dough;  
    String sauce;  
    String cheese;  
  
    public abstract void prepare();  
  
    public void bake(){ ... }  
    public void cut(){ ... }  
    public void box(){ ... }  
    public void getName(){  
        return name;  
    }  
}
```

← Clase base para las pizzas

← Cada pizza tendrá un nombre, un tipo de masa y una salsa

← Provee implementaciones para los métodos

```
public class NYStyleCheesePizza extends Pizza{  
  
    public NYStyleCheesePizza(){  
        name = "NY Style Sause and Cheese Pizza";  
        dough = "Thin Crust Dough";  
        sauce = "Marinara sauce";  
    }  
  
    public void prepare(){ ... }  
  
}
```

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



- Se tiene flexibilidad en la creación de las pizzas
- Las franquicias tienen que respetar los procesos definidos
- Pero, se deja a las franquicias la libertad de elegir sus ingrediente
- Se pueden crear familias de ingredientes con factories

```
public interface PizzaIngredientFactory{
```

```
    public Dough createDough();  
    public Sauce createSauce();  
    public Cheese createCheese();  
    public Veggie [] createVeggies();  
    public Pepperoni createPepperoni();
```

```
}
```

Se define un método por cada ingrediente

ThinCrustDough

FreshClams

MarinaraSauce

ReggianitoCheese

Ingredientes para New York

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



- Se tiene flexibilidad en la creación de las pizzas
- Las franquicias tienen que respetar los procesos definidos
- Pero, se deja a las franquicias la libertad de elegir sus ingrediente
- Se pueden crear familias de ingredientes con factories

1 Se crea un factory para cada región

2 Se implementa el conjunto de ingredientes a ser utilizados en cada factory

3 Se inserta este código en nuestro PizzaStore

ThinCrustDough	FreshClams
MarinaraSauce	ReggianitoCheese

Ingredientes para New York

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



```
public class NYPizzaIngredientFactory implements PizzaIngredientFactory{

    public Dough createDough(){
        return new ThinCrustDough();
    }

    public Sauce createSauce(){
        return new MarinaraSauce();
    }

    public Cheese createCheese(){
        return new ReggianoCheese();
    }

    public Veggie [] createVeggies() { ... }
    public Pepperoni createPepperoni(){ ... }

}
```

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



```
public abstract class Pizza{
```

```
    String name;  
    Dough dough;  
    Sauce sauce;  
    Cheese cheese;  
    Pepperoni pepperoni;  
    Veggies [] veggies;
```

```
    public abstract void  
    prepare();
```

```
        public void bake(){ ... }  
        public void cut(){ ... }  
        public void box(){ ... }  
        public void getName(){  
            return name;  
        }  
    }
```

- Cuando se hizo el primer Factory Method, se tenía NYCheesePizza y ChicagoCheesePizza.
- Esas clases lo único que tenían diferente era el uso de los ingredientes.
- Ahora, con el Factory para los ingredients no hace falta tener dos clases

```
public class CheesePizza extends Pizza{
```

```
    PizzaIngredientFactory ingredientFactory;
```

```
    public CheesePizza(PizzaIngredientFactory  
    ingredientFactory) {  
        this.ingredientFactory = ingredientFactory;  
    }
```

```
    public void prepare(){  
        dough = ingredientFactory.createDough();  
        sauce = ingredientFactory.createSauce();  
        cheese = ingredientFactory.createCheese();  
    }
```

Para hacer la pizza ahora
necesitamos un factory
que nos determine los
ingredientes

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



```
public abstract class Pizza{
```

```
    String name;  
    Dough dough;  
    Sauce sauce;  
    Cheese cheese;  
    Pepperoni pepperoni;  
    Veggies [] veggies;
```

```
    public abstract void  
    prepare();
```

```
    public void bake(){ ... }  
    public void cut(){ ... }  
    public void box(){ ... }  
    public void getName(){  
        return name;  
    }  
}
```

- Cuando se hizo el primer Factory Method, se tenía NYCheesePizza y ChicagoCheesePizza.
- Esas clases lo único que tenían diferente era el uso de los ingredientes.
- Ahora, con el Factory para los ingredients no hace falta tener dos clases

```
sauce = ingredientFactory.createSauce();
```

A la pizza no le importa
qué factory es, siempre
cuando sea un
IngredientFactory

Va a devolver la salsa
correspondiente con el
IngredientFactory

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



```
public class NYPizzaStore extends PizzaStore{  
  
    PizzaIngredientFactory ingredientFactory = new NYPizzaIngredientFactory();
```

```
    public Pizza createPizza(String type){  
        Pizza pizza;  
        if(type.equals("cheese"){  
            pizza = new CheesePizza(ingredientFactory);  
            pizza.setName("New York Style Cheese Pizza");  
        }else if(type.equals("greek"){  
            pizza = new GreekPizza(ingredientFactory);  
            ...  
        }else if(type.equals("pepperoni"){  
            pizza = new PepperoniPizza(ingredientFactory);  
            ...  
        }else if(type.equals("clam"){  
            pizza = new ClamPizza();  
            ...  
        }else if(type.equals("veggie"){  
            pizza = new VeggiePizza(ingredientFactory);  
            ...  
        }  
        return pizza;  
    }  
}
```

Tiene el IngredientFactory
para construir las pizzas
correspondientes a NY

Reciben el factory para poder
construir los ingredientes
correspondientes

Factory – version 3

Pizzeria Panucci: Y los ingredientes?



```
public class NYPizzaStore extends PizzaStore{

    PizzaIngredientFactory ingredientFactory = new NYPizzaIngredientFactory();

    public Pizza createPizza(String type){
        Pizza pizza;
        if(type.equals("cheese"){
            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("New York Style Cheese Pizza");
        }else if(type.equals("greek"){
            pizza = new GreekPizza(ingredientFactory);
            ...
        }else if(type.equals("pepperoni"){
            pizza = new PepperoniPizza(ingredientFactory);
            ...
        }else if(type.equals("clam"){
            pizza = new ClamPizza();
            ...
        }else if(type.equals("veggie"){
            pizza = new VeggiePizza(ingredientFactory);
            ...
        }
        return pizza;
    }
}
```

- Se desacopla aún más el código del factory que crea los productos.
- Esto permite más flexibilidad en la creación de productos para diferentes contextos
- Del Factory abstracto se derivan los Factories concretos que producen los mismos productos, pero con distintas implementaciones
- Para los clientes estos cambios son transparentes

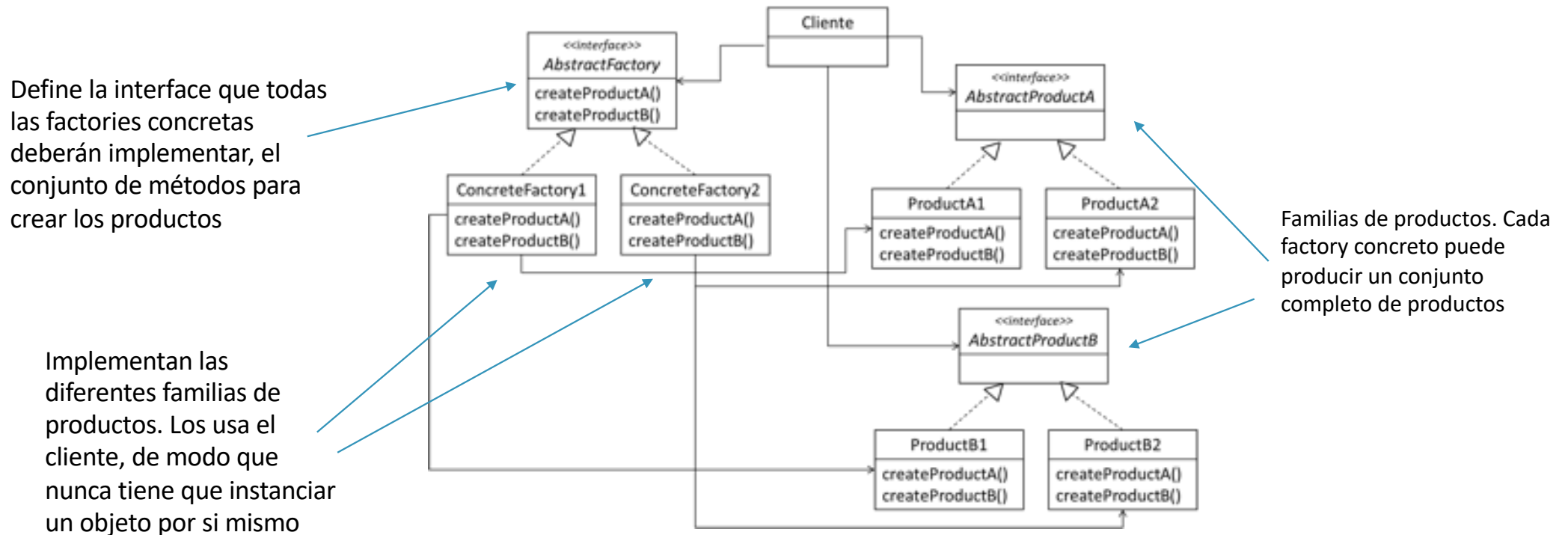
Abstract Factory

Abstract Factory prove una interfaz para la creación de familias de objetos relacionados sin especificar sus clases concretas

El cliente queda desacoplado de los productos concretos que son creados

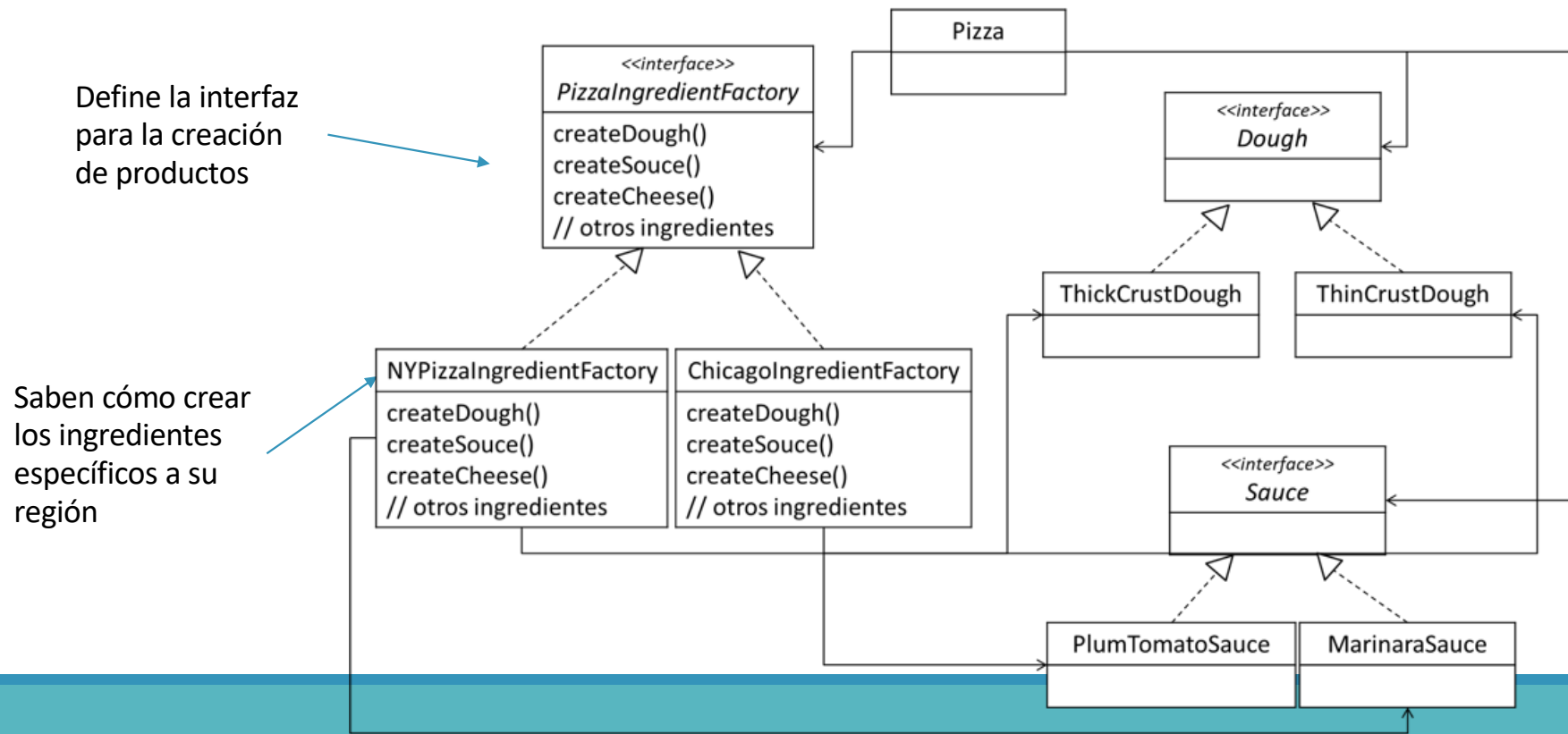
Abstract Factory

Abstract Factory prove una interfaz para la creación de familias de objetos relacionados sin especificar sus clases concretas.



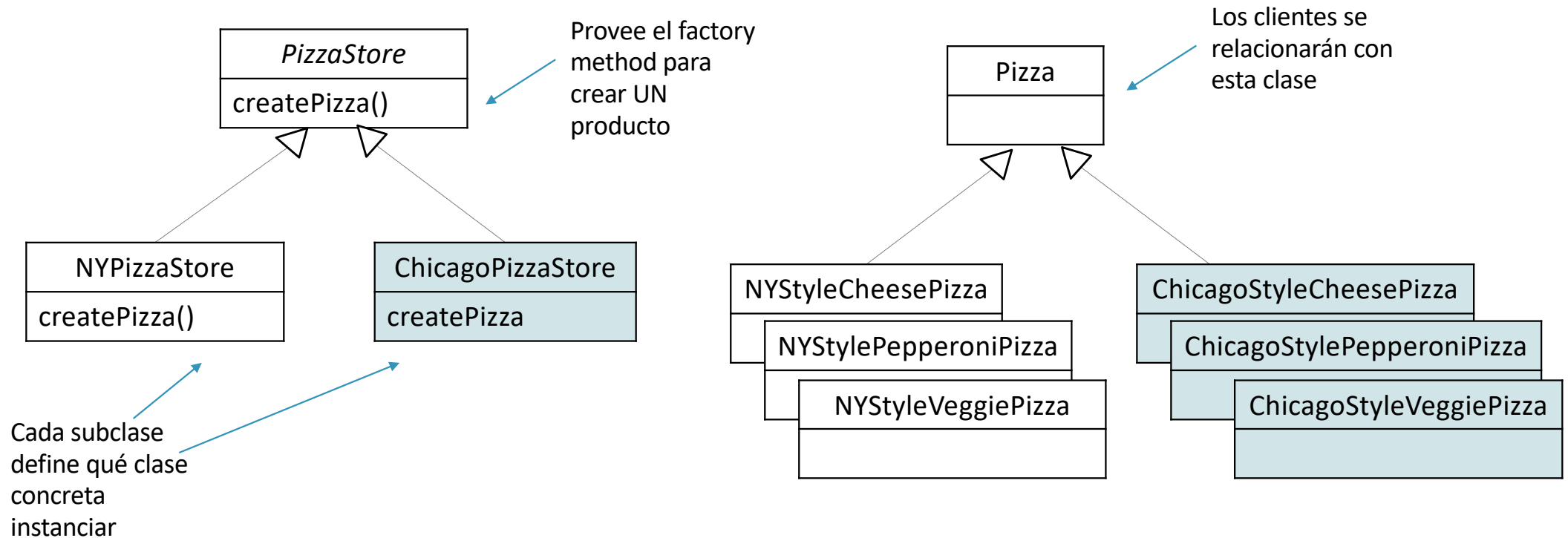
Abstract Factory

Abstract Factory prove una interfaz para la creación de familias de objetos relacionados sin especificar sus clases concretas.



Abstract Factory vs. Factory Method

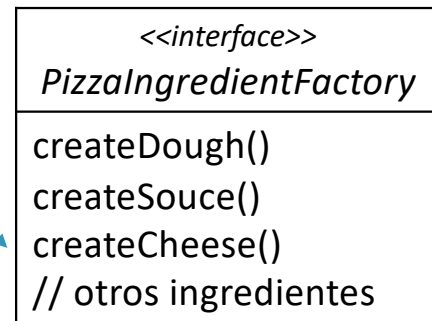
- Los métodos de los abstract factories son implementados como factory methods
- Cada método en la interfaz es responsable de crear un producto concreto, y se implementa una subclase del abstract factory para proveer esas implementaciones



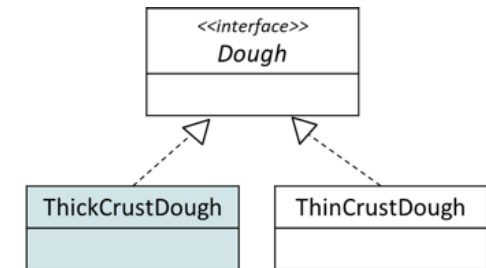
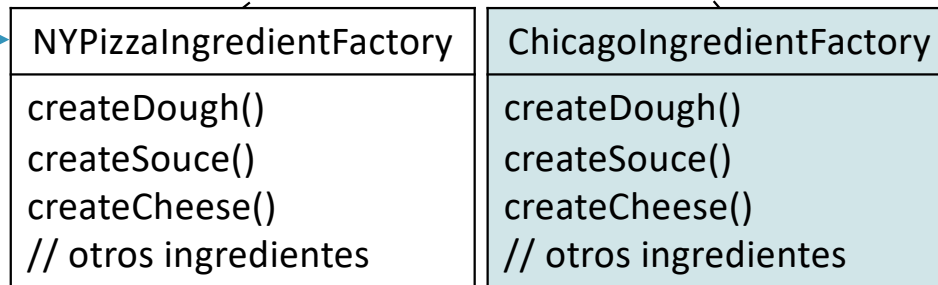
Abstract Factory vs. Factory Method

Abstract factory
porque se quiere
poder crear
familias de
productos.

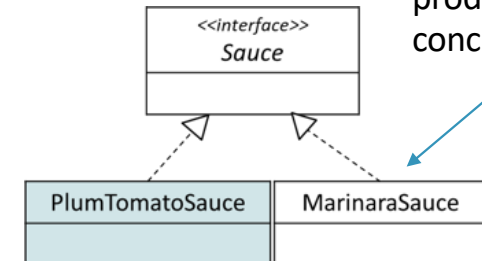
La interfaz para
crear los
productos



Deciden qué
ingredientes
concretos
utilizar



Cada ingrediente
representa un
producto
concreto a crear



Factory (recap)

Abstract Factory

- Provee una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

Factory Method

- Define una interfaz para crear un objeto, pero deja que las subclasses decidan la clase a instanciar.
 - Difiere la instanciación a las subclasses.
-
- Todos los factory encapsulan la creación de objetos
 - Factory Method se basa en la herencia. La creación de objetos es delegada a las subclasses, los cuales implementan el factory method para crear los objetos

Principios OO

- Depender de abstracciones, no de clases concretas.
-
- AbstractFactory se basa en la composición. La creación de objetos es implementada en métodos expuestos en la interface
 - Todos los patrones factory favorecen el desacoplado mediante la reducción de las dependencias entre la aplicación y las clases concretas
 - Factory Method intenta diferir la instanciación de objetos a las subclasses
 - Abstract Factory crea familias de objetos relacionados sin tener que depender de las clases concretas

Patrones everywhere

Strategy

Decorator

Factory

Singleton

Adapter

Command

Visitor

Template



Singleton

Único en su especie

- Permite que se cree una y solo una instancia de una clase. Para qué?
 - Objetos de los que solo se necesita uno: thread pools, caches, manejo de preferencias, loggers, ...
 - Si de estos hubiera más de una instancia, es probable que se encuentren problemas como comportamientos incorrecto, sobre-uso de recursos, o resultados inconsistentes

```
public class Singleton{  
    private static Singleton uniqueInstance;  
    //otros atributos
```

Tenemos atributo estático que contiene la única instancia de la clase Singleton

```
    private Singleton() {}
```

El constructor es declarado privado para que solo esta clase pueda accederlo

```
    public static Singleton getInstance(){  
        if (uniqueInstance == null)  
            uniqueInstance = new Singleton();  
        return uniqueInstance  
    }  
    // otros métodos
```

Este método nos permite crear y obtener la instancia correspondiente

```
}
```

Singleton

Único en su especie

- Permite que se cree una y solo una instancia de una clase. Para qué?
 - Objetos de los que solo se necesita uno: thread pools, caches, manejo de preferencias, loggers, ...
 - Si de estos hubiera más de una instancia, es probable que se encuentren problemas como comportamientos incorrecto, sobre-uso de recursos, o resultados inconsistentes

```
public static Singleton getInstance(){  
    if (uniqueInstance == null)  
        uniqueInstance = new Singleton();  
  
    return uniqueInstance  
}
```

Atributo estático

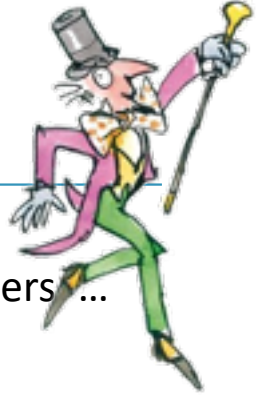
Si la instancia es null, es que todavía no ha sido creada

Si no existe, se crea la instancia con el constructor privado y se la asigna al atributo. Si nunca la necesitamos, nunca se va a crear. Lazy

Acá ya tenemos una instancia, o por haberla creado recién o porque ya existía y la podemos retornar

Singleton y la fábrica de chocolate

Qué pasa si más de una instancia de
boiler es creada en el Sistema?
Cómo se podría evitar?



- Permite que se cree una y solo una instancia de una clase. Para qué?
 - Objetos de los que solo se necesita uno: thread pools, caches, manejo de preferencias, loggers ...
 - Si de estos hubiera más de una instancia, es probable que se encuentren problemas como comportamientos incorrecto, sobre-uso de recursos, o resultados inconsistentes

```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;  
  
    public ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }  
  
    public void fill() {  
        if (isEmpty()) {  
            empty = false;  
            boiled = false;  
            // hacemos la mezcla  
        }  
    }  
}
```

Se empieza con el boiler
vacío

Para llenar el boiler, se
cheque que no esté lleno.
Una vez que está llengo,
se ajustan los flags

```
public void drain() {  
    if (!isEmpty() && isBoiled()) {  
        // sacamos la mezcla  
        empty = true;  
    }  
}
```

Para vaciar el boiler, se
cheque que esté lleno
y que ya haya hervido.
Una vez que se vacia,
se ajustan los flags

```
public void boil() {  
    if (!isEmpty() && !isBoiled()) {  
        // hervimos  
        boiled = true;  
    }  
}
```

Para hervir, el boiler tiene
que estar lleno y no tuvo
que haberse hervido antes.

```
public boolean isEmpty() { return empty; }  
public boolean isBoiled() { return boiled; }
```

}

Singleton

y la fábrica de chocolate



```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;
```

```
    //TODO: Atributo nuevo?
```

```
    public ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }
```

```
    //TODO: Método nuevo?
```

```
    // el resto de los métodos sigue igual...
```

```
}
```


Singleton

y la fábrica de chocolate



```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;  
  
    private static ChocolateBuilder uniqueInstance;  
  
    private ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }  
  
    public static ChocolateBuilder getInstance(){  
        if (uniqueInstance == null)  
            uniqueInstance = new ChocolateBuilder();  
        return uniqueInstance;  
    }  
  
    // el resto de los métodos sigue igual...  
}
```

Singleton

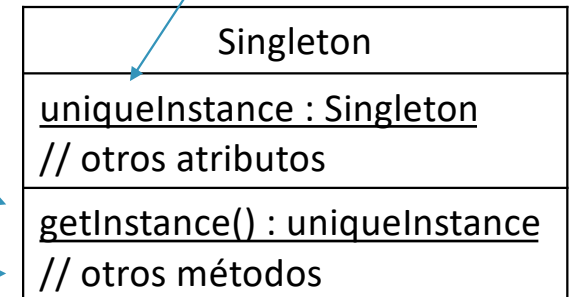
Garantiza que haya una única instancia de una clase y provee un único punto de acceso a la misma

- Permite que una clase maneje una instancia de si mismo
- Se prohíbe que otra clase cree una instancia de esa clase
- Se provee un punto de acceso global a la instancia
- La creación de la instancia es Lazy, lo que es importante para objetos que consumen muchos recursos

Siguen siendo clases de propósito general, con lo que seguro va a tener otros métodos y atributos

El método es static, con lo que puede accedérselo desde cualquier parte del código.

El atributo `uniqueInstance` guarda la única instancia



Singleton

y la fábrica de chocolate



- Nuestro Boiler Singleton venía bien... hasta que pasaron cosas.
- De alguna forma se obtuvieron 2 instancias distintas de boiler y el método fill() comenzó a llenar el boiler cuando ya había otro batch!
- Qué pudo haber pasado?

Problema de SINCRONIZACIÓN!!

```
public static ChocolateBuilder getInstance(){  
    if (uniqueInstance == null)  
        uniqueInstance = new ChocolateBuilder();  
    return uniqueInstance;  
}
```

Podría pasar que 2 threads entren al mismo tiempo al getInstance(), consulten por null, los dos pasen y terminen creando y devolviendo instancias separadas

Qué hacemos?

Singleton

y la fábrica de chocolate



- Nuestro Boiler Singleton venía bien... hasta que pasaron cosas.
- De alguna forma se obtuvieron 2 instancias distintas de boiler y el método fill() comenzó a llenar el boiler cuando ya había otro batch!
- Qué pudo haber pasado?

Problema de SINCRONIZACIÓN!!

```
public static synchronized ChocolateBuilder getInstance(){  
    if (uniqueInstance == null)  
        uniqueInstance = new ChocolateBuilder();  
    return uniqueInstance;  
}
```

Pero, la única vez que el **synchronized** es relevante, es la primera vez que se tiene que ejecutar el método (y crear la instancia).

Luego, ya no se necesita, con lo que la sincronización se convierte en overhead!

Con el **synchronized** se fuerza a que cada thread tenga que esperar antes de poder entrar al método.

Qué hacemos?

Singleton

y sincronización... opciones

1 Si la performance de getInstance() no es crítica, no hacer nada

2 No usar invocación lazy

- Si la instancia se va a usar siempre, no esperar a que se la pida para crearla, sino crearla desde el principio
- La JVM garantiza que va a haber una única

```
public class Singleton{
    private static Singleton uniqueInstance;

    public static Singleton getInstance(){
        if (uniqueInstance == null)
            uniqueInstance = new Singleton();
        return uniqueInstance;
    }
}
```

```
public class Singleton{
    private static Singleton uniqueInstance = new Singleton();

    public static Singleton getInstance(){
        return uniqueInstance;
    }
}
```

Singleton

y sincronización... opciones

- 1 Si la performance de getInstance() no es crítica, no hacer nada
- 2 No usar invocación lazy
- 3 Usar doble chequeo para evitar el uso de synchronized
 - Primero se chequea si la instancia está creada, sino, ahí se sincroniza
 - Solo se utiliza la sincronización en la primera entrada

```
public class Singleton{
    private volatile static Singleton uniqueInstance;

    public static Singleton getInstance(){
        if (uniqueInstance == null)
            synchronized (uniqueInstance){
                if (uniqueInstance == null)
                    uniqueInstance = new Singleton();
            }
        return uniqueInstance;
    }
}
```

Singleton

y sincronización... opciones

- 1 Si la performance de getInstance() no es crítica, no hacer nada
- 2 No usar invocación lazy
- 3 Usar doble chequeo para evitar el uso de synchronized
 - Primero se chequea si la instancia está creada, sino, ahí se sincroniza
 - Solo se utiliza la sincronización en la primera entrada

```
public class Singleton{  
    private volatile static Singleton uniqueInstance;  
  
    public static Singleton getInstance(){  
        if (uniqueInstance == null)  
            synchronized (uniqueInstance){  
                if (uniqueInstance == null)  
                    uniqueInstance = new Singleton();  
            }  
        return uniqueInstance;  
    }  
}
```

Mantiene consistencia entre los threads.
Cuestiones de almacenamiento y acceso

Chequear si ya se creó la instancia. Si no hay,
entramos en la sincronización

La sincronización solo se utiliza la primera
vez, cuando la instancia todavía no fue
creada

Singleton (recap)

Singleton

- Asegura que hay una única instancia de una clase.
 - Provee un punto global de acceso a dicha instancia
-
- Para la implementación se tiene un constructor privado, un atributo estático y un método estático
 - Si se necesita multi-thread, ver los requerimientos de performance, para elegir la mejor opción
 - Tener cuidado con el uso de múltiples class-loaders

Patrones everywhere (recap)

Strategy

Decorator

Factory

Singleton

Adapter

Command

Visitor

Template



Patrones - como catálogo

Los patrones de diseño son soluciones a problemas generales que suelen aparecer durante el desarrollo

	Propósito			
Alcance		Creacional	Estructural	Comportamental
	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Brigde Composite Decorator Façade Proxy	Chain of Responsibility Command Iterator Mediator Memento Fliweight Observer State Strategy Visitor

Patrones

Aspectos cambiantes

Propósito	Design Pattern	Aspectos que pueden variar
Creacional	Abstract Factory	Familias de productos
	Builder	Cómo crear un composite
	Factory Method	Subclase del objeto que es instanciado
	Prototype	Clase del objeto que es instanciado
	Singleton	La única instancia de la clase

Propósito	Design Pattern	Aspectos que pueden variar
Estructural	Adapter	Interfaz de un objeto
	Bridge	Implementación de un objeto
	Composite	Estructura y composición de un objeto
	Decorator	Responsabilidades de un objeto sin hacer subclases
	Façade	Interfaz de un subsistema
	Flyweight	Costo de almacenamiento de los objetos
	Proxy	Cómo se accede a un objeto

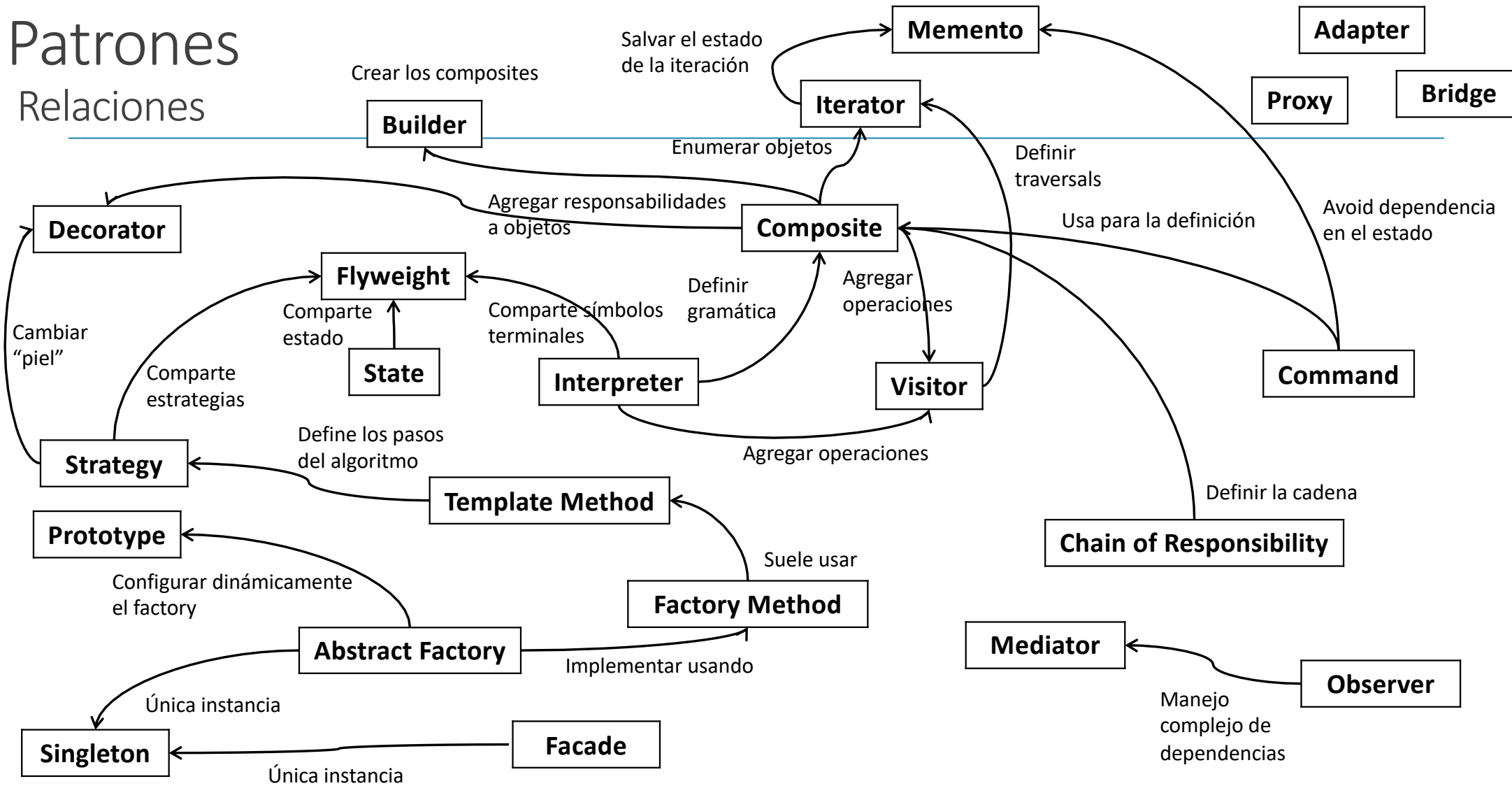
Patrones!

Aspectos cambiantes

Propósito	Design Pattern	Aspectos que pueden variar
Comportamental	Chain of responsibility	Objetos que pueden completar un pedido
	Command	Cuando y cómo los pedidos son completados
	Interpreter	Gramática e interpretación de un lenguaje
	Iterator	Cómo un elemento agregado es recorrido y accedido
	Mediator	Qué objetos y cómo interactúan
	Memento	Qué información privada de un objeto es almacenada por fuera de él
	Observer	Número de objetos que dependen de otro, cómo los objetos dependientes se mantienen actualizados
	State	Estado de un objeto
	Strategy	Algoritmo
	Template Method	Pasos de un algoritmo
	Visitor	Operaciones que pueden ser aplicadas a objetos sin modificar sus clases

Patrones

Relaciones



Cómo elegir los patrones?

- 1 Considerar cómo los patrones ayudan a resolver problemas
 - Qué problema tengo?
 - Para qué sirve el patrón?
 - Qué hace?
- 2 Analizar la utilidad e intención de los patrones
 - Cuál podría ser útil?
 - Cuál sirve para lo que quiero?
- 3 Analizar las interrelaciones entre los patrones
- 4 Analizar las causas de un re-diseño
 - Analizar qué puede forzar a un cambio de diseño
 - Ver qué patrones puede ayudar a evitar ese re-diseño
- 5 Considerar qué puede cambiar en el diseño
 - Lo opuesto al punto anterior
 - Qué es lo que queríamos que pueda cambiar sin forzar a un re-diseño?



Excederse con el uso de patrones a veces es contraproducente, porque agrega más complejidad al diseño (que luego hay que entender, mantener, testear, etc.)

Material de referencia

